

Security Review

Cantina Competition

ROYCO DAY



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	<code>EntryPoint</code> reentrancy: a market's oracle <code>poke()</code> double-spends commingled escrow	4
3.2	Informational	9
3.2.1	The Accountant retroactively applies an unadapted curve to the first active epoch	9
3.2.2	A late first synchronization after fixed-term expiry irreversibly erases post-expiry adaptive premium history	11
3.2.3	Liquidity premium keeps paying genesis dead shares after full LP exit, locking whole tokens of senior yield per cycle	16
3.2.4	<code>OPTIMIZED / MAX</code> LPT redemptions resolve on gross queued shares, settle on net	24
3.2.5	Each oracle replacement setter reads the other oracle, one shared outage freezes every operation with no on-chain recovery	34
3.2.6	A released fixed term is re-stamped in full by the next downtick, freezing all tranche redemptions indefinitely	37



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

A competition provides a broad evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While competitions endeavor to identify and disclose all potential security issues, they cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities, therefore, any changes made to the code would require an additional security review. Please be advised that competitions are not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above matrix. High severity findings represent the most critical issues that must be addressed immediately, as they either have high impact and high likelihood of occurrence, or medium impact with high likelihood.

Medium severity findings represent issues that, while not immediately critical, still pose significant risks and should be addressed promptly. These typically involve scenarios with medium impact and medium likelihood, or high impact with low likelihood.

Low severity findings represent issues that, while not posing immediate threats, could potentially cause problems in specific scenarios. These typically involve medium impact with low likelihood, or low impact with medium likelihood.

Lastly, some findings might represent improvements that don't directly impact security but could enhance the codebase's quality, readability, or efficiency (Gas and Informational findings).



2 Security Review Summary

From Aug 10th to Aug 19th Cantina hosted a competition based on [royco-day](#). The participants identified a total of **7** issues in the following risk categories:

- High Risk: 0
- Medium Risk: 1
- Low Risk: 0
- Gas Optimizations: 0
- Informational: 6



3 Findings

3.1 Medium Risk

3.1.1 `EntryPoint` reentrancy: a market's oracle `poke()` double-spends commingled escrow

Severity: Medium Risk.

Context: (No context files were provided by the reviewer).

Submitted by [MalikX](#).

Target: `src/entrypoint/RoycoDayEntryPoint.sol`, at commit `fa6d2497`. The whole file is post-audit: it did not exist at the Hexens review commit `c006198`.

In-scope impact: (A) funds sent to an address not specified by a whitelisted party, and (B) funds permanently locked / unrecoverable.

Summary: I have confirmed that a single deposit escrowed in `RoycoDayEntryPoint` is settled twice inside one transaction by an unprivileged attacker: `executeDeposit` reads the request into memory, calls out to the market's collateral-asset oracle (`poke()`, declared non-view) before it writes the escrow back, and the contract carries no reentrancy guard, so an attacker-supplied oracle reenters `executeDeposit` in that window and the same request settles a second time. The entry point is one singleton holding every user's deposit escrow commingled per asset, so the second settlement pulls its collateral out of other users' escrow, mints the attacker double the tranche shares, and leaves the raided user unable to execute or cancel their own request.

I have confirmed this against the project's own deployment fixture with a fork-free, mock-free-on-the-path test. Escrowing `X = 40e18` for the attacker and `100e18` for an unrelated depositor, then executing the attacker's single request once, the entry point pays out `80e18` (`2X`) of collateral for that one `X`-escrow request, the attacker holds `80e18` (`2X`) of junior shares, and the `100e18` depositor can no longer execute or cancel. A second test running the identical flow with the oracle un-armed settles once and the depositor recovers in full, so the loss is attributable to the reentry and not to the harness. I have not needed to model a second market: the mechanism is a single-market instance of a per-asset commingling that spans every market on the singleton entry point.

This is the unguarded-sibling of a guarded contract. The market's other user-facing entry contract, the kernel, inherits `ReentrancyGuardTransient` and applies `nonReentrant` to every state-mutating entrypoint; the entry point, which is the contract that actually custodies escrow, inherits and applies neither. The root cause is a checks-effects-interactions violation on shared escrow: the escrow state write is ordered after an untrusted external call, on a contract with no reentrancy lock.

Root cause: `executeDeposit` reads the request into memory, makes an external call, and only then mutates the escrow:

```
// src/entrypoint/RoycoDayEntryPoint.sol
function executeDeposit(address _user, uint256 _requestNonce, TRANCHE_UNIT
↳ _assetsToDeposit) external whenNotPaused restricted returns (uint256) {
//...
    DepositRequest memory request =
    ↳ $.userToNonceToDepositRequest[_user][_requestNonce]; // 170: read into
    ↳ MEMORY
//...
    _validateRequestExecution(_requestNonce, request.baseRequest, config);
    ↳ // 175: EXTERNAL CALL to oracle.poke()
//...
    IRoycoDayKernel(config.kernel).syncTrancheAccountingFor(config.trancheType);
    ↳ // 182: kernel entered here (first nonReentrant)
//...
    if (assetsLeftToDeposit == ZERO_TRANCHE_UNITS) {
        delete $.userToNonceToDepositRequest[_user][_requestNonce];
        ↳ // 196-204: escrow written HERE, after the call
    } else { ... }
```



```
//...
(trancheSharesMinted, protocolFeeShares) = _depositWithShareForfeiture(...);
↪ // 212: pulls _assetsToDeposit from the EntryPoint's balance
//...
}
```

The external call at line 175 is `_validateRequestExecution` reaching `_pokeOracle`, which resolves the oracle live off the kernel and calls it:

```
// src/entrypoint/RoycoDayEntryPoint.sol _pokeOracle (~612)
lastUpdatedAtTimestamp = IRoycoPriceOracle(IRoycoDayKernel(_config.kernel).getCollateralAssetOracle()).poke().toUint32();
↪
```

`poke()` is non-view, so a deployer-chosen oracle contract runs arbitrary code at this point:

```
// src/interfaces/IRoycoPriceOracle.sol
function poke() external returns (uint256 updatedAt);
```

The guard is on the wrong contract: The kernel, the other user-facing entry contract in a Day market, is guarded. The entry point, which custodies the escrow, is not. This is a per-contract enforcement inconsistency, verified at commit `fa6d2497`:

Contract	Role	Inherits ReentrancyGuard Transient	nonReentrant on mutating entrypoints
RoycoDayKernel	market core, user-facing	Yes (RoycoDayKernel.sol:26)	Yes: syncTrancheAccountingFor , inkindDeposit , inkindRedeem , lptDeposit MultiAsset , lptRedeemMulti Asset , reinvest Liquidity Premium
RoycoDayEntryPoint	async escrow custody, user-facing	No (not in it, RoycoUUPSBase , RoycoBase , or RoycoAuth)	No (nowhere)

The exact primitive the kernel uses is absent from the periphery that holds user funds.

The kernel's guard also cannot cover this window even in principle. The reentry at line 175 fires before the kernel is entered at line 182: the reentrant `executeDeposit` completes its own kernel round-trip and returns before the outer call ever touches the kernel, so the kernel's `nonReentrant` lock is free the entire time. A guard on the kernel does not protect an escrow write that happens on the entry point, before the kernel is reached.

The double settlement: With no lock, the reentrant call reads the same intact request from storage (the outer call has not yet reached the `delete` at line 196), settles it, and deletes it. When control returns, the outer call proceeds on its stale in-memory copy of the request and settles it a second time. `_depositWithShareForfeiture` (line 632) calls `tranche.deposit` (line 644), which pulls `_assetsToDeposit` from the entry point's asset balance (RoycoVaultTranche.sol:72) on each



The attacker's oracle is deliberately honest on price. It returns a fixed non-zero WAD price and a fresh timestamp on every read, identical to the fixture's own mock, so the market's accounting is never distorted and no price or NAV is misreported. Its only added behavior is a single reentrant `executeDeposit`, latched to fire once. This keeps the finding cleanly separated from oracle-manipulation: the oracle is a reentrancy vector, not a bad price.

Reachability: Every step of the attack is open to an address holding no role, and I have verified each link against the deployed configuration.

Market deployment is permissionless. `RoycoFactory.executeMarketDeployment` is restricted but bound to `PUBLIC_ROLE` in `script/deploy/core/DeployCore.s.sol` (~113-118), where the in-code comment states "Market deployment is PERMISSIONLESS." The collateral oracle is a deployer-supplied parameter, not an admin setting: `MarketParams.collateralAssetOracle` (`RoycoDayBalancerV3MarketDeploymentTemplate.sol:166`) flows into the kernel's init via `RoycoDayKernelInitParams{ collateralAssetOracle: _params.collateralAssetOracle }` (:538), and `executeMarketDeployment` routes it there. At deploy it is validated only for `code.length > 0` (`MarketDeploymentValidationLogic.sol:120`, then :276-284), and the kernel additionally pins only `oracle.COLLATERAL_ASSET() == market.collateralAsset` (`RoycoDayKernel.sol:_setCollateralAssetOracle`). There is no registry, allowlist, or check on the oracle's behavior. The admin `setCollateralAssetOracle` only changes an existing market's oracle; it is not on the deploy path and the attack does not use it.

The deployer's tranche configuration reaches the singleton entry point permissionlessly. The deployment carries the deployer-supplied `entryPointTrancheConfigs` (`RoycoDayBalancerV3MarketDeploymentTemplate.sol:442-445`) into `modifyTrancheConfigs` on the one entry point through `RoycoFactoryGatekeeper` (:86-103), because the factory holds `ADMIN_ENTRY_POINT_ROLE`. The attacker sets `gateByOracleUpdate = true` in that config, which is what makes the entry point poke the oracle inside `_validateRequestExecution`.

The LP surface is public. `requestDeposit`, `executeDeposit`, and the cancels are bound to `PUBLIC_ROLE` in `script/deploy/core/DeployPeriphery.s.sol` (~99-107). The entry point itself holds the LP roles it needs to call `tranche.deposit` on users' behalf.

In the proof of concept, the whole deployer-side sequence is driven by a single roleless address (`ROLELESS_DEPLOYER`). It queues the tranche configs and calls `executeMarketDeployment`, both under one `vm.startPrank` of that address. `queueTrancheConfigs` is not a production function; it exists only on the test-mock registration template and carries no access control, standing in for the deployer-supplied `MarketParams.entryPointTrancheConfigs` that the real template reads from its own params. The proof of concept then asserts the `gateByOracleUpdate = true` config actually landed on the singleton entry point through that roleless call. The attacker and the innocent depositor are likewise plain addresses with no role.

The only genuinely privileged step anywhere near this is template enablement: the fixture calls `registerTemplate` (`RoycoFactory.sol:82`, `restricted = ADMIN_FACTORY_ROLE`) to enable a deployment template, exactly as the design intends. The README states that templates are admin-registered and enabled, and that anyone then deploys through them. The attacker consumes an already-enabled template; they never register one. Everything else the fixture does is the protocol's own role-graph wiring, and none of it is part of the attack.

Impact: This lands on both in-scope impacts.

For (A), a normal depositor's escrowed collateral is pulled out of the entry point to mint the attacker's tranche shares, which is value leaving toward a recipient the depositor never named. Tranche shares are a direct, redeemable claim on the pooled collateral through `RedemptionLogic.inkindRedeem`, so the extracted collateral is realizable, not notional. This is a direct loss of user funds; Royco's severity page places direct theft or loss of user funds at High.

For (B), after the raid the entry point holds less of the asset than the depositor's request requires. `cancelDepositRequest` transfers the full `request.assets` back (:555) and `executeDeposit` needs to pull `request.assets` for the deposit, and both revert on the short balance. There is no



partial cancel and no rescue path on the entry point. The depositor cannot unilaterally recover, and the attacker can repeat the drain, so for the raided user the loss is effectively permanent.

On magnitude: the target is pre-launch, so there is no live TVL to price. The native unit of loss here is the collateral asset. Each execution nets the attacker their own escrow `X` in extra shares, drawn from the pool, and the attack is repeatable within a block, so the loss is bounded only by the total pending deposit escrow of that collateral asset across all markets on the singleton entry point at attack time. Worst case and realistic case coincide, because the double-spend is deterministic and permissionless: any concurrent pending escrow of a shared asset is at risk, with no dependence on price movement, timing, or a privileged actor.

The blast radius is per-asset, not per-market. Because the entry point is a single singleton commingling escrow by asset, a depositor in a completely honest, unrelated market is drained the same way the moment their market shares a collateral asset with the attacker's market. The proof of concept demonstrates the mechanism within one market (an innocent senior-tranche depositor drained through a junior-tranche attack), which is the single-market instance of that general case.

What this is not. It is not a way to withdraw to an external address the entry point would otherwise block, and it is not a break of the blacklist or sanctions screening; those still run on the paths they run on. It is not contingent on any oracle mispricing, and it does not require the attacker to hold any market role or to control any admin key.

Why this is in scope: The finding is not oracle manipulation. No price is misreported and no NAV is misvalued; the proof of concept oracle prices identically to an honest one, and `poke()` is used only as a reentrancy vector into an unguarded contract. The vulnerability is in `RoycoDayEntryPoint`, not in any price.

It is not a whitelisted or trusted party behaving maliciously. The attacker is an untrusted `PUBLIC_ROLE` market deployer, the oracle is an unvalidated deployment parameter rather than a trusted component, and the victim is an ordinary depositor. The proof of concept uses no role in the attack; the only privileged action in the fixture is by-design template enablement, which the attacker consumes rather than performs.

It is not an incorrect amount sent to a whitelisted party under the reversible carve-out. The victim's loss is not a mis-sized payout to them; it is their escrow taken by a third party and rendered unwithdrawable, so it is neither directed to them nor reversible by them.

It is not centralization or an admin risk. No admin action participates in the attack, and the proof of concept demonstrates this end to end: the reentrant oracle is a real contract wired in through the kernel's deploy-time `initialize` parameter rather than the admin setter, the tranche gate config is applied by a roleless address through the permissionless `executeMarketDeployment`, and the attacker and victim hold no role.

Not a duplicate: The entire entry point postdates Hexens' audited commit `c006198`; `git diff --stat c006198 HEAD -- src/entrypoint` is `+1370 / -0`, so the file did not exist at review time. Hexens' three entry-point findings (batch revert isolation, LT forfeiture ignoring the idle premium, the blacklist bypass) and Olympix's Medium (blacklist receiver bypass) are all different issues and are fixed in the current code. The project's own reentrancy test, `test/concrete/Tranches/Test_RedeemReentrancyWindow.t.sol` with `MockReentrancyProbe`, exercises a receiver reentering the kernel from an ERC20 transfer hook, where `nonReentrant` catches it. The oracle-poke-into-the-unguarded-entry-point path is the sibling that is not covered, and there is no invariant suite on the entry point at all.

Recommended fix: Add a reentrancy guard to `RoycoDayEntryPoint` and apply it to `executeDeposit`, `executeRedemption`, the batch wrappers, `requestDeposit`, `requestRedemption`, and the cancel paths. The kernel already inherits `ReentrancyGuardTransient` and uses `nonReentrant`; the entry point should do the same, and a transient-storage guard is cheap and correct here. Independent of the guard, move the escrow state write ahead of the oracle poke and any other external call: the `delete` and assets writeback at `:196-204`, and the redemption twin at `:383-391`, should execute before `_validateRequestExecution` runs, so the request cannot be re-read intact mid-flight. Either change closes this specific double-spend; both together restore checks-effects-interactions on the escrow, which is the property the current ordering breaks.



Royco: Fixed in commit [9764c9e2](#).

3.2 Informational

3.2.1 The Accountant retroactively applies an unadapted curve to the first active epoch

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Submitted by IronsideSec.

Summary: The Accountant opens its premium clock without checkpointing the adaptive YDM's independent clock. At the first later synchronization, the YDM adapts over zero seconds while the Accountant applies that unadapted output to the entire elapsed epoch, materially misallocating yield.

Description: The Accountant and adaptive YDM maintain independent clocks, but the Accountant starts its accrual epoch without starting the YDM's adaptation epoch.

On the first accrual, `RoycoDayAccountant._accruePremiumYieldShares` sets `lastYieldShareAccrualTimestamp` and `lastPremiumPaymentTimestamp`, then returns without calling either YDM. Every additional call in the same block also returns because `elapsed == 0`:

```
if (lastUpdate == 0) {
    $.lastYieldShareAccrualTimestamp = uint32(block.timestamp);
    $.lastPremiumPaymentTimestamp = uint32(block.timestamp);
    return (0, 0);
}
//...
if (elapsed == 0) return (...);
```

Meanwhile, `AdaptiveCurveYDM_V2.initializeYDMForMarket` explicitly deletes the model's `lastAdaptationTimestamp`. On the first later call, `BaseAdaptiveCurveYDM._yieldShare` treats the model's elapsed time as zero:

```
uint256 elapsed = lastAdaptationTimestamp == 0
    ? 0
    : block.timestamp - lastAdaptationTimestamp;
```

It therefore returns and stores the unadapted initial curve. The Accountant, however, multiplies that returned rate by its own complete nonzero interval. The full active epoch is retroactively priced as though the curve never adapted.

This arises organically when launch/seeding establishes active capital in the same block in which the Accountant clock opens. It is also reproducible on supported live model replacement: `setJuniorTrancheYDM` and `setLiquidityProviderTrancheYDM` initialize a replacement—which resets its clock—but never checkpoint it to the Accountant epoch.

The mismatch occurs within a single interval: when the model clock begins at the same timestamp as the Accountant clock, the YDM adapts over that interval and produces the corresponding average rate.

Impact Explanation: Impact is Medium. The clock mismatch can materially misallocate the first epoch's premium. In the proof of concept, the live model pays 29 NAV instead of the 95.75 NAV produced by the properly adapted curve, a 66.75 NAV allocation difference. The error is bounded by the senior gain available for distribution and affects yield rather than principal custody.

Likelihood Explanation: Likelihood is Medium. Same-block launch seeding can create the mismatch without a privileged action. A material result additionally requires an adaptive model, off-target utilization, a meaningful quiet first interval, and a later gain. Model replacement recreates the mismatch but requires the configured Accountant authority.



Under Cantina's severity matrix, Medium likelihood and Medium impact produce Medium severity.

Proof of Concept: This full-market test uses the production Accountant, Kernel, tranches, deposit paths, adaptive V2, the protocol-wide 90% target, and the shipped snUSD JT curve (11%, 11%, 31%). A separately deployed and correctly primed reference model supplies the counterfactual rate.

Create `test/AuditFirstEpochCanonicalPoC.t.sol` with:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import {MarketFuzzTestBase} from "../utils/MarketFuzzTestBase.sol";
import {MarketParamsConfig} from "../utils/FixtureTypes.sol";
import {defaultParams} from "../utils/MarketParams.sol";
import {cellA} from "../utils/TokenConfigs.sol";
import {AdaptiveCurveYDM_V2} from "../src/ydm/AdaptiveCurveYDM_V2.sol";
import {IRoycoDayAccountant} from "../src/interfaces/IRoycoDayAccountant.sol";
import {MarketState, SyncedAccountingState} from "../src/libraries/Types.sol";
import {toUint256} from "../src/libraries/Units.sol";

contract AuditFirstEpochCanonicalPoC is MarketFuzzTestBase {
    AdaptiveCurveYDM_V2 actualCurve;
    AdaptiveCurveYDM_V2 correctlyClockedReference;
    uint256 constant START = 1_000_000;
    uint256 constant ELAPSED = 2_030_488;

    function setUp() public override {
        vm.warp(START);
        MarketParamsConfig memory p = defaultParams();
        p.minCoverageWAD = uint64(0.09e18);
        p.maxJTYieldShareWAD = uint64(1e18);
        p.maxLPTYieldShareWAD = 0;
        p.stProtocolFeeWAD = 0;
        p.jtProtocolFeeWAD = 0;
        p.jtYieldShareProtocolFeeWAD = 0;
        p.lptYieldShareProtocolFeeWAD = 0;
        p.fixedTermDurationSeconds = 0;
        p.dustTolerance = 0;
        p.jtYdmKind = 2;
        p.jtCurve = [uint64(0.11e18), uint64(0.11e18), uint64(0.31e18)];
        p.lptYdmKind = 0;
        p.lptCurve = [uint64(0), uint64(0), uint64(0)];
        p.targetUtilizationWAD = uint64(0.9e18);
        _deployMarket(cellA(), p);
        actualCurve = AdaptiveCurveYDM_V2(address(jtYdm));

        // Same-block production deposit paths establish 99% coverage utilization.
        _seedMarket(1000e18, 100e18);

        correctlyClockedReference = new AdaptiveCurveYDM_V2(
            0.9e18, 0.0001e18, 1e18, 100e18 / uint256(365 days)
        );
        correctlyClockedReference.initializeYDMForMarket(
            uint64(0.11e18), uint64(0.11e18), uint64(0.31e18)
        );
        correctlyClockedReference.yieldShare(MarketState.PERPETUAL, 0.99e18);
    }

    function test_firstEpochUsesUnadaptedShippedCurve() public {
        IRoycoDayAccountant.RoycoDayAccountantState memory beforeState =
            account.get();
        (, uint32 actualYdmTimestampBefore,) =
            actualCurve.accountantToCurve(address(account));
        assertEq(beforeState.lastYieldShareAccrualTimestamp, uint32(START));
    }
}
```



```

assertEq(actualYdmTimestampBefore, 0);

vm.warp(START + ELAPSED);
collateralAssetOracle.setUpdatedAt(block.timestamp);
priceFeed.setUpdatedAt(block.timestamp);
uint256 correctShare = correctlyClockedReference.previewYieldShare(
    MarketState.PERPETUAL, 0.99e18
);
applySTPNL(1000); // +10%: 110 gain, of which 100 is gross senior gain
SyncedAccountingState memory synced = _sync();

uint256 actualPremium = toUint256(synced.jtEffectiveNAV) - 100e18 - 10e18;
(uint64 actualTarget, uint32 actualYdmTimestampAfter, ) =
    actualCurve.accountantToCurve(address(accountant));
emit log_named_uint("correctly clocked shipped-curve average share",
    → correctShare);
emit log_named_uint("actual first-epoch JT premium", actualPremium);
emit log_named_uint("correct first-epoch JT premium", 100e18 * correctShare
    → / 1e18);
assertEq(actualTarget, 0.11e18, "live YDM did not adapt across the first
    → epoch");
assertEq(actualYdmTimestampAfter, uint32(START + ELAPSED));
assertEq(actualPremium, 29e18, "initial 29% output was retroactively
    → applied");
assertGt(correctShare, 0.85e18);
}
}

```

Run from the repository root at `fa6d24971a5b1993e4d067fc223c85c8139346c0` :

```
forge test --match-path test/AuditFirstEpochCanonicalPoC.t.sol -vv
```

Observed output:

```

Ran 1 test for test/AuditFirstEpochCanonicalPoC.t.sol:AuditFirstEpochCanonicalPoC
[PASS] test_firstEpochUsesUnadaptedShippedCurve() (gas: 355238)
Logs:
  correctly clocked shipped-curve average share: 9575000000000000000
  actual first-epoch JT premium: 29000000000000000000
  correct first-epoch JT premium: 95750000000000000000

Suite result: ok. 1 passed; 0 failed; 0 skipped

```

Recommendation: Whenever the Accountant opens an accrual epoch, atomically checkpoint both adaptive models at the same timestamp and current market state/utilizations while discarding rate accrual. Do the same immediately after successful YDM replacement. A dedicated `checkpointForMarket(state, utilization)` interface is safer than overloading a pricing call. Add regressions for JT and LPT, V1 and V2, above/below target utilization, same-block genesis deposits, long quiet first epochs, and live replacement.

Royco: Acknowledged.

Cantina Managed: Acknowledged.

3.2.2 A late first synchronization after fixed-term expiry irreversibly erases post-expiry adaptive premium history

Severity: Informational.

Context: (No context files were provided by the reviewer).

Submitted by [Ironsidesec](#).



Summary: The Accountant accrues the entire elapsed interval using the stored `FIXED_TERM` state before checking whether its deterministic deadline already passed. The adaptive YDM freezes its curve but still advances its clock to the late synchronization time, so the post-expiry interval is never adapted or accrued as `PERPETUAL`, materially redistributing later senior yield.

Description: A fixed term ends at a deterministic timestamp. If an accrual interval crosses that timestamp, accounting should split the interval into a frozen fixed-term segment and an adaptive perpetual segment. Royco instead labels the whole interval with the state stored at its beginning.

In `src/accountant/RoycoDayAccountant.sol:140-178`, `preOpSyncTrancheAccounting` accrues both YDMs **before** it previews and commits the market-state transition:

```
(twJTYieldShareAccruedWAD, twLPTYieldShareAccruedWAD) =
    _accruePremiumYieldShares();
(state, initialMarketState, premiumsPaid, jtImpermanentLossErased) =
    _previewSyncTrancheAccounting(
        _collateralNAV,
        twJTYieldShareAccruedWAD,
        twLPTYieldShareAccruedWAD
    );
```

The accrual function in `src/accountant/RoycoDayAccountant.sol:593-621` passes the stale persisted state to each model for the entire elapsed interval:

```
uint256 jtYieldShareWAD = Math.min(
    IYDM($.jtYDM).yieldShare(
        $.lastMarketState,
        coverageUtilizationWAD
    ),
    $.maxJTYieldShareWAD
);
//...
$.lastYieldShareAccrualTimestamp = uint32(block.timestamp);
```

Only afterward does `src/accountant/RoycoDayAccountant.sol:528-562` check `fixedTermEndTimestamp <= block.timestamp` and return the market to `PERPETUAL`.

Adaptive curves deliberately do not adapt when the supplied state is `FIXED_TERM` (`src/ydm/base/BaseAdaptiveCurveYDM.sol:95-127`). Nevertheless, the stateful V2 call always stores `lastAdaptationTimestamp = uint32(block.timestamp)` in `src/ydm/AdaptiveCurveYDM_V2.sol:145-158`. Therefore, for fixed-term deadline `T` and first later synchronization `S`:

1. The Accountant supplies `FIXED_TERM` for all time since the preceding accrual, including `[T, S]`;
2. The model keeps the old anchor and prices the entire interval with the frozen curve;
3. The model and Accountant both advance their clocks to `S`; and.
4. The later market-state transition records `PERPETUAL`, but no later call can recover the lost `[T, S]` adaptation or premium accrual.

This is not merely a stale view label. It changes the time-weighted premium entitlement already accumulated and delays the anchor from which all future adaptation starts. At above-target utilization the suppression can underpay JT/LPT premium recipients; at below-target utilization the direction can reverse.

The condition occurs whenever no transaction synchronizes at the exact deadline. No exact-boundary keeper is guaranteed, and some deadlines will not coincide with a producible block timestamp. Several production recipes combine a nonzero term with Adaptive V2—for example the `srRoyUSDC` configuration uses a seven-day term and adaptive JT/LPT curves (`script/deploy/templates/royco-day-balancer-v3/markets/Market_SrRoyUSDC.sol:40-61`).



The proof of concept compares two otherwise identical full-market branches: one has no synchronization until 30 days after expiry; the reference branch checkpoints the deterministic deadline. It reports a 270.608046729172716348 NAV JT delta on the same later gain. The fixture uses production contracts and snUSD-shaped adaptive anchors with a seven-day term, a 100% host cap, and non-canonical coverage and liquidation values to isolate the accounting effect rather than estimate a specific shipped market's loss. The result demonstrates material premium redistribution.

Impact Explanation: Impact is Medium. A late synchronization charges the post-expiry interval under the fixed-term state and advances the YDM clock past that interval, irreversibly changing later premium allocation between the senior and recipient tranches. The proof of concept measures a 270.608 NAV branch difference, but the error redistributes yield rather than consuming principal.

Likelihood Explanation: Likelihood is Medium. Any adaptive market with a nonzero fixed term follows this path when its first synchronization after the exact deadline is late. Normal inactivity or user-chosen timing is sufficient, while an earlier synchronization shortens or prevents the gap.

Under Cantina's severity matrix, Medium likelihood and Medium impact produce Medium severity.

Proof of Concept: Test file: test/YDMM01PoC.t.sol

Tested revision: fa6d24971a5b1993e4d067fc223c85c8139346c0

Prerequisites: initialized submodules; no RPC URL is required.

The complete compiling test below deploys the production Accountant, Kernel, AccessManager, and Adaptive V2 implementation through the repository's market fixture. It initializes the YDM clock before entering `FIXED_TERM`, snapshots the state, and compares a lazy first sync against an exact-deadline checkpoint.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import {MarketFuzzTestBase} from "../test/utils/MarketFuzzTestBase.sol";
import {MarketParamsConfig} from "../test/utils/FixtureTypes.sol";
import {defaultParams} from "../test/utils/MarketParams.sol";
import {cellA} from "../test/utils/TokenConfigs.sol";
import {AdaptiveCurveYDM_V2} from "../src/ydm/AdaptiveCurveYDM_V2.sol";
import {MarketState, SyncedAccountingState} from "../src/libraries/Types.sol";
import {toUint256} from "../src/libraries/Units.sol";

contract AuditYdmFixedExpiryGapFullPoC is MarketFuzzTestBase {
    AdaptiveCurveYDM_V2 internal model;
    uint256 constant START = 1_000_000;
    uint256 constant TERM = 7 days;
    uint256 constant POST_EXPIRY = 30 days;

    function setUp() public override {
        vm.warp(START);
        MarketParamsConfig memory p = defaultParams();
        p.minCoverageWAD = uint64(0.2e18);
        p.coverageLiquidationUtilizationWAD = 6.4667e18;
        p.maxJTYieldShareWAD = uint64(1e18);
        p.maxLPTYieldShareWAD = 0;
        p.stProtocolFeeWAD = 0;
        p.jtProtocolFeeWAD = 0;
        p.jtYieldShareProtocolFeeWAD = 0;
        p.lptYieldShareProtocolFeeWAD = 0;
        p.fixedTermDurationSeconds = uint24(TERM);
        p.fixedTermGracePeriodSeconds = 0;
        p.dustTolerance = 0;
        p.jtYdmKind = 2;
        p.jtCurve = [uint64(0.11e18), uint64(0.11e18), uint64(0.31e18)];
        p.lptYdmKind = 0;
        p.lptCurve = [uint64(0), uint64(0), uint64(0)];
    }
}
```




```
p.targetUtilizationWAD = uint64(0.9e18);
_deployMarket(cellA(), p);
model = AdaptiveCurveYDM_V2(address(jtYdm));
_seedMarket(1000e18, 300e18);

// Open the YDM clock while perpetual, then create a covered loss. The
→ post-loss
// utilization is 520%, capped to 100% by the YDM, and the market enters
→ FIXED_TERM.
vm.warp(START + 1);
_freshOracleTime(block.timestamp);
_sync();
applySTPNL(-2000);
_freshOracleTime(block.timestamp);
SyncedAccountingState memory entered = _sync();
assertEq(uint8(entered.marketState), uint8(MarketState.FIXED_TERM));
}

function _freshOracleTime(uint256 t) internal {
    collateralAssetOracle.setUpdatedAt(t);
    priceFeed.setUpdatedAt(t);
}

function _payGainOneSecondLater(uint256 t) internal returns (uint256 jtNAV) {
    vm.warp(t + 1);
    // Move 0.8 -> 1.2 of the original price. Since both branches already erased
    → the
    // fixed-term IL, the same 520 NAV gain is split/premium-paid from identical
    → base state.
    applySTPNL(5000);
    _freshOracleTime(block.timestamp);
    jtNAV = toUint256(_sync().jtEffectiveNAV);
}

function test_lazyExpiryDiscardsPostExpiryAdaptationAndUnderpaysJT() public {
    uint32 end = accountant.getState().fixedTermEndTimestamp;
    assertEq(uint256(end), START + 1 + TERM);
    uint256 comparisonTime = uint256(end) + POST_EXPIRY;
    uint256 snapshot = vm.snapshotState();

    // Lazy path: no checkpoint at expiry. Thirty days later _accrue still
    → supplies the
    // stored FIXED_TERM state, so the YDM freezes *and restamps* across all 30
    → post-expiry days.
    vm.warp(comparisonTime);
    _freshOracleTime(block.timestamp);
    _sync();
    (uint64 lazyYT, uint32 lazyClock, ) =
    → model.accountantToCurve(address(accountant));
    uint128 lazyTW = accountant.getState().twJTYieldShareAccruedWAD;
    assertEq(lazyYT, 0.11e18);
    assertEq(lazyClock, uint32(comparisonTime));
    uint256 lazyJTNAV = _payGainOneSecondLater(comparisonTime);

    vm.revertToState(snapshot);

    // Reference path: checkpoint the deterministic wall-clock expiry, then let
    → the same
    // utilization run for the same 30 post-expiry days in PERPETUAL state.
    vm.warp(end);
    _freshOracleTime(block.timestamp);
    _sync();
    assertEq(uint8(accountant.getState().lastMarketState),
    → uint8(MarketState.PERPETUAL));
```



```

vm.warp(comparisonTime);
_freshOracleTime(block.timestamp);
_sync();
(uint64 timelyYT, uint32 timelyClock,,) =
    ↪ model.accountantToCurve(address(accountant));
uint128 timelyTW = accountant.getState().twJTYieldShareAccruedWAD;
assertEq(timelyYT, 1e18);
assertEq(timelyClock, uint32(comparisonTime));
uint256 timelyJTNAV = _payGainOneSecondLater(comparisonTime);

emit log_named_uint("lazy-expiry curve target", lazyYT);
emit log_named_uint("timely-expiry curve target", timelyYT);
emit log_named_uint("missing TW yield-share history", timelyTW - lazyTW);
emit log_named_uint("JT NAV after lazy-expiry premium", lazyJTNAV);
emit log_named_uint("JT NAV after timely-expiry premium", timelyJTNAV);
emit log_named_uint("JT underpayment", timelyJTNAV - lazyJTNAV);
assertGt(timelyTW, lazyTW);
assertGt(timelyJTNAV - lazyJTNAV, 100e18);
    }
}

```

Run from a clean checkout at `fa6d24971a5b1993e4d067fc223c85c8139346c0` :

```

FOUNDRY_TEST=test/YDMM01PoC.t.sol forge test \
--match-contract AuditYdmFixedExpiryGapFullPoC \
--match-test test_lazyExpiryDiscardsPostExpiryAdaptationAndUnderpaysJT -vv

```

Observed output for `test/YDMM01PoC.t.sol` :

```

Ran 1 test for test/YDMM01PoC.t.sol:AuditYdmFixedExpiryGapFullPoC
[PASS] test_lazyExpiryDiscardsPostExpiryAdaptationAndUnderpaysJT() (gas: 741963)
Logs:
  lazy-expiry curve target: 110000000000000000
  timely-expiry curve target: 100000000000000000
  missing TW yield-share history: 173016000000000000000000
  JT NAV after lazy-expiry premium: 214999968718765566973
  JT NAV after timely-expiry premium: 485608015447938283321
  JT underpayment: 270608046729172716348
Suite result: ok. 1 passed; 0 failed; 0 skipped
Ran 1 test suite in 119.19ms: 1 tests passed, 0 failed, 0 skipped (1 total tests)

```

Expected behavior is that `[deadline, lateSync]` is accrued and adapted as `PERPETUAL` regardless of whether a keeper calls exactly at the deadline. Actual behavior leaves the lazy anchor at `0.11e18`, advances its clock to the late sync, and produces `270.608046729172716348` less JT NAV than the boundary-checkpointed reference on the same later gain.

Recommendation: Split every elapsed accrual interval at `fixedTermEndTimestamp` when:

```
lastYieldShareAccrualTimestamp < fixedTermEndTimestamp < block.timestamp
```

Accrue/freeze only through the deadline, then accrue an adaptive `PERPETUAL` segment from the deadline to the current block. Preview and execution must perform the identical split.

The current YDM derives elapsed time from `block.timestamp`, so two calls in the same transaction cannot reconstruct two historical endpoints. Add an explicit `yieldShareAt(timestamp/elapsed, state, utilization)` checkpoint API or move the authoritative cumulative index into the Accountant. Add tests comparing no-boundary-call and exact-boundary-call branches for both adaptive versions, both sides of target utilization, and each nonzero-term production recipe.



Royco: Acknowledged. This is accepted behavior; keepers sync the market as soon as it exits the fixed term to replicate the proposed split.

Cantina Managed: Acknowledged.

3.2.3 Liquidity premium keeps paying genesis dead shares after full LP exit, locking whole tokens of senior yield per cycle

Severity: Informational.

Context: (No context files were provided by the reviewer).

Submitted by *Jornason*.

Summary: The production deployment template locks `1e12` LPT shares at `0x00000000000000000000000000000000dEaD` at genesis, so the liquidity-provider tranche's supply can never reach zero. Once the last real LP exits through ordinary redemptions, every further liquidity premium — a carve-out of senior yield, configured at up to 50% in the officially deployed FalconX/APYX/SrRoyUSDC registry markets — is minted to a tranche whose only shareholder is the dead address. The carved value is fully backed, permanently unrecoverable, grows every gain cycle (measured: 261.95 whole tokens in a single post-exit cycle), and cannot be captured by later depositors. The protocol keeps paying for liquidity that no longer exists, and pays it into a black hole.

Description: The security guarantee broken: a yield carve-out paid for a service (liquidity provision) must accrue to claimants who can actually redeem it, and must cease — or stay attributable to a recoverable surface — when no such claimant exists. Here, after a full real-LP exit, the premium keeps accruing to shares that can never act, permanently diverting senior yield to `0x...dEaD`.

Root cause, two in-scope `src/` components interacting:

1. **Genesis dead-share lock** — `src/factory/templates/RoycoDayBalancerV3MarketDeploymentTemplate.sol` : `DEAD_SHARES = 1e12` and `DEAD_ADDRESS = 0x...dEaD` (lines 222-225); `_seedPool` requires the genesis mint to be at least `DEAD_SHARES` and transfers them to the dead address (lines 487-488). This is the deliberate anti-inflation fix for a prior-review issue, and it makes `liquidityProviderTranche.totalSupply() >= 1e12` forever.
2. **Supply-blind premium accrual** — `src/accountant/RoycoDayAccountant.sol`:496 computes `lptLiquidityPremium = stGain * lptYieldShare` with no check on whether any real LPT supply exists, and `src/libraries/logic/FeeAndLiquidityPremiumLogic.sol`:90 converts the premium into senior tranche shares credited to `lptOwnedSeniorTrancheShares` (line 59). Because the dead shares keep the supply nonzero, these mints price normally and do NOT floor to zero — the value materializes in full, owned pro-rata by a supply that is 100% dead after a real-holder exit.

Attack/loss walkthrough (all ordinary, whitelisted, non-manipulative operations):

1. A premium-enabled market runs normally — this is the officially deployed shape: the protocol's own registry markets FalconX, APYX and SrRoyUSDC all deploy `maxLPTYieldShareWAD = 0.5e18` with nonzero LPT curves (`script/deploy/templates/royco-day-balancer-v3/markets/Market_{FalconX,APYX,SrRoyUSDC}.sol`); only the SnUSD baseline disables it (`maxLPTYieldShareWAD = 0`).
2. The real LP(s) fully exit via `redeemMultiAsset` — a normal wind-down/migration. They are paid fairly; the exiting LPs lose nothing. Supply is now exactly `DEAD_SHARES`.
3. The next senior gain syncs: the accountant carves the LPT share of `stGain` (up to 50% at the official cap) and mints it as senior shares owned by the dead-only tranche. Proof of concept test B measures 261.95 whole tokens of NAV locked in this single post-exit cycle.
4. This repeats every gain cycle, forever. No public, privileged, or future transaction can pay the accumulated claim to anyone: the only shareholder is `0x...dEaD`, and a later depositor mints at fair pricing against a `totalValue` that includes the locked claim, so their full exit returns at most their own deposit (proof of concept test B: deposit 1000, recover 999.99999999999966, capture exactly 0 of the 261.95 locked pile, which survives the cycle intact).



The control probe (test A) pins the defect precisely: under the premium-disabled SnUSD shape, the same full exit strands only wei-level dust and nothing ever accrues post-exit. The whole-token loss exists exactly at the intersection the official registry markets occupy: premium enabled × dead-only supply.

Prior-awareness disclosure. The dead-share lock itself is a documented design choice (it fixes a prior-review finding in which a near-zero-supply LPT residual could be *captured by a later depositor* — a transfer to a later claimant). This finding reports the opposite, post-fix consequence that issue did not cover: value accruing to a claimant that can never exist, driven not by a static residual but by *ongoing premium accrual* to the dead-only tranche. The fix prevents capture; it does not prevent accrual, and we found no documented waiver indicating the protocol accepts paying perpetual premiums to `0x...dEaD`. A related terminal-redemption residual on the ST/JT side (underpayment via the redemption-side virtual share) is a distinct mechanism with a distinct fix and is reported separately: there the claim surface reaches zero supply; here the supply is permanently nonzero but permanently ownerless, and no redemption-side change can address it.

Impact Explanation: Impact = Medium, matching the in-scope impact “Funds permanently locked / unrecoverable”. The premium carved after a full real-LP exit is physically backed (senior tranche shares / pool assets in kernel custody) yet no transaction by any party — public, whitelisted, or protocol — can ever pay it out: it is locked permanently, not delayed. Quantified by the proof of concept at moderate market scale: 261.95 whole tokens locked in a single post-exit gain cycle (test B, exact wei-level assertions), and 103 whole tokens stranded in one cycle under a dust-scale real supply (test C). The loss recurs on every subsequent gain sync and only grows. The economically harmed party is the senior side: the premium is carved out of `stGain`, so yield that would otherwise accrue to senior holders is burned to `0x...dEaD` on every cycle after the exit. High is not claimed because the material loss mode requires a preceding full real-LP exit (or dust-scale real supply), and the per-cycle magnitude is bounded by the configured LPT share of senior yield rather than the whole market NAV.

Likelihood Explanation: Likelihood = Medium. Every step is ordinary protocol behavior by whitelisted parties — deposits, multi-asset redemptions, gains, syncs — with no oracle/NAV/share-price manipulation and no malicious configuration: the premium-enabled shape is the protocol’s own officially deployed registry configuration (FalconX/APYX/SrRoyUSDC at `maxLPTYieldShareWAD = 0.5e18`), so the “deployer misconfiguration” and “pathological creator parameters” objections do not apply. Full LP exits are a normal part of a market’s lifecycle (wind-downs, migrations, liquidity consolidation), and once the dead-only state is reached the leak is deterministic on every gain. Not rated higher because a market with continuous real LP presence only loses the negligible `1e12 / totalSupply` dead fraction of each premium, so the material window opens only when real supply fully exits or decays to dust scale.

Proof of Concept: Coded proof of concept (mandatory-POC rule). The test deploys the real Kernel, Accountant, tranches, factory/registration path, and role graph at the scope commit `fa6d24971a5b1993e4d067fc223c85c8139346c0`; only external tokens and oracles are mocked. The genesis dead-share lock that the fixture skips is replicated exactly as the production template performs it (1e12 LPT shares transferred to `0x...dEaD` after the seed). File: `test/poc/Test_LPTDeadSharePremiumLock.t.sol` (drops into the scope repository’s existing `test/` tree, using its `test/Utils` fixtures; run with the repository’s default Foundry profile).

Run:

```
forge test --match-path test/poc/Test_LPTDeadSharePremiumLock.t.sol -vv
```

Expected: `3 passed; 0 failed; 0 skipped`. Observed key values (6-decimal ERC4626 collateral shares, 18-decimal quote 1:1 with NAV; exact base-unit assertions):

```
test_PremiumDisabledShape_NoPremiumEver_ExitStrandsOnlyDust: (control: official
↪ SnUSD shape)
  A_premium_accrued_nav: 0 (no premium can ever
↪ accrue)
```



A_supply_after_exit:	1000000000000	(= DEAD_SHARES
↳ exactly)		
A_residual_nav:	1000000000001	(wei-level dust only)
A_postexit_premium_accrued_nav:	0	(dead-only tranche
↳ earns nothing)		
test_PremiumEnabled_FullExit_PremiumLocksToDead_NoCapture:		(official
↳ FalconX/APYX/SrRoyUSDC shape)		
B_preexit_premium_accrued_nav:	31000000000000000000	(310 tokens, 31% of
↳ the senior gain)		
B_postexit_supply:	1000000000000	(REACHABLE: real
↳ supply fully exited)		
B_postexit_claim_nav:	1155000000171	(tranche emptied to
↳ dust by the exit)		
B_postexit_premium_accrued_nav:	261950000310000000000	(261.95 tokens paid
↳ AFTER the exit)		
B_locked_claim_nav:	261950001518474953929	(100% owned by dead
↳ shares)		
B_late_lp_deposit_nav:	1000000000000000000000	
B_late_lp_out_nav:	99999999999996219002	(later depositor
↳ recovers only their own deposit)		
B_locked_claim_after_late_cycle:	261950001533469420485	(the locked pile
↳ survives; capture = 0)		
test_TinyRealSupply_PremiumInflation_StrandsWholeTokens:		(amplifier:
↳ dust-scale real supply)		
C_real_shares:	2000000000000	(2e12 real vs 1e12
↳ dead)		
C_claim_before_exit_nav:	310000002999999999999	(one 31% premium
↳ cycle inflates the claim)		
C_terminal_payout_nav:	20666666866659777775	
C_terminal_residual_nav:	103333334371812131312	(103 whole tokens,
↳ ownerless, one cycle)		
C_residual_after_flat_sync_nav:	103333334371812131312	(flat sync does not
↳ release it)		

The three tests verify, respectively: (A) the mechanism is gated by the premium-enabled configuration — under the premium-disabled official baseline a full exit strands only dust; (B) under the officially deployed premium-enabled shape, an ordinary full LP exit compresses real supply to the dead-share floor, after which the very next gain pays 261.95 whole tokens of senior yield to the dead-only tranche, and a later depositor cannot capture any of it; (C) with a dust-scale real supply, a single premium cycle strands 103 whole tokens of ownerless claim on the terminal exit.

Full test source:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import { Math } from
↳ "../../lib/openzeppelin-contracts/contracts/utils/math/Math.sol";
import { WAD } from "../../src/libraries/Constants.sol";
import { LPT_LP_ROLE } from "../../src/factory/Roles.sol";
import { AssetClaims, SyncedAccountingState } from "../../src/libraries/Types.sol";
import { toUint256 } from "../../src/libraries/Units.sol";
import { DayMarketTestBase } from "../utils/DayMarketTestBase.sol";
import { FixtureCell, MarketParamsConfig } from "../utils/FixtureTypes.sol";
import { zeroLiquidityParams } from "../utils/MarketParams.sol";
import { cellB } from "../utils/TokenConfigs.sol";

/**
 * @notice Public PoC: after the last real LP exits a premium-enabled market, the
 * ↳ liquidity premium keeps accruing
```



```

*         to the genesis dead shares (1e12 LPT shares locked at 0x...dEaD by the
↳ production deployment template),
*         so every further senior-yield payment is carved into a claim no one can
↳ ever redeem – whole tokens per
*         cycle under the officially deployed market shape – and later depositors
↳ cannot capture the locked pile.
* @dev Three probes:
*     A) The official SnUSD baseline shape (maxLPTYieldShareWAD == 0): no LPT
↳ premium can ever accrue, so a full
*         real-holder exit strands only the wei-level floor( $A \cdot S / (S + 1)$ ) dust – the
↳ mechanism is gated by the
*         premium-enabled configuration.
*     B) A premium-enabled configuration (matches the non-SnUSD official registry
↳ markets FalconX/APYX/SrRoyUSDC,
*         which deploy maxLPTYieldShareWAD = 0.5e18 with nonzero LPT curves):
↳ ordinary redemptions CAN compress the
*         real LPT supply to zero (supply == DEAD_SHARES) while senior yield keeps
↳ paying the liquidity premium –
*         100% of it accrues to the dead-only tranche. A later depositor mints at
↳ fair virtual-shares pricing and
*         recovers at most their own deposit: the accrued pile is locked, not
↳ captured.
*     C) Amplifier: a dust-scale real supply (same order as DEAD_SHARES) plus one
↳ premium cycle inflates the
*         tranche claim, and the terminal exit strands the dead shares' pro-rata
↳ fraction – whole tokens per cycle.
* @dev Fixture: cell B – 6-decimal ERC4626 collateral shares, 18-decimal quote (so
↳ quote wei == NAV wei 1:1).
*     The fixture deploys tranches directly, so the production genesis dead-share
↳ lock is replicated by transferring
*         1e12 LPT shares to the template's DEAD_ADDRESS after the seed.
*/
contract Test_LPTDeadSharePremiumLock is DayMarketTestBase {
    using Math for uint256;

    /// @dev Mirrors RoycoDayBalancerV3MarketDeploymentTemplate.DEAD_SHARES /
    ↳ DEAD_ADDRESS
    uint256 internal constant DEAD_SHARES = 1e12;
    address internal constant DEAD_ADDRESS =
    ↳ 0x000000000000000000000000000000000000000000000000000000000000dEaD;

    /// @dev The official SnUSD baseline shape: zero minimum liquidity, LPT premium
    ↳ disabled, permanently perpetual
    function _officialSnUsdShapeParams() internal pure returns (MarketParamsConfig
    ↳ memory p) {
        p = zeroLiquidityParams(); // minLiquidityWAD = 0, maxLPTYieldShareWAD = 0,
        ↳ lptCurve = [0,0,0]
        p.fixedTermDurationSeconds = 0; // SnUSD: stable market, no fixed term
    }

    /// @dev The premium-enabled shape the non-SnUSD official registry markets
    ↳ deploy (FalconX/APYX/SrRoyUSDC carry
    ///     maxLPTYieldShareWAD = 0.5e18 with nonzero LPT curves), simplified: zero
    ↳ fees, zero JT share, the LPT
    ///     share pinned at 31% (every official LPT curve tops out at 0.31e18 at
    ↳ full utilization), zero minimum
    ///     liquidity so the exit path needs no ST-exit choreography (minLiquidity
    ↳ only gates timing, not
    ///     reachability: ST exits are unrestricted in PERPETUAL, and the
    ↳ multi-asset exit heals the requirement
    ///     in-flow by redeeming the pool's senior leg)
    function _premiumEnabledParams() internal pure returns (MarketParamsConfig
    ↳ memory p) {
        p = zeroLiquidityParams();

```



```
p.fixedTermDurationSeconds = 0;
p.stProtocolFeeWAD = 0;
p.jtProtocolFeeWAD = 0;
p.jtYieldShareProtocolFeeWAD = 0;
p.lptYieldShareProtocolFeeWAD = 0;
p.maxJTYieldShareWAD = 0;
p.jtCurve = [uint64(0), uint64(0), uint64(0)];
p.maxLPTYieldShareWAD = 0.31e18;
p.lptCurve = [uint64(0.31e18), uint64(0.31e18), uint64(0.31e18)];
}

function _lptClaimNav() internal view returns (uint256) {
    return toUint256(liquidityProviderTranche.totalAssets().nav);
}

function _lptIdlePremiumStShares() internal view returns (uint256) {
    return kernel.getState().lptOwnedSeniorTrancheShares;
}

/// @dev Replicates the production genesis dead-share lock the fixture skips
/// ↳ (template locks 1e12 at seed)
function _lockDeadShares() internal {
    vm.prank(LPT_PROVIDER);
    liquidityProviderTranche.transfer(DEAD_ADDRESS, DEAD_SHARES);
    assertEq(liquidityProviderTranche.balanceOf(DEAD_ADDRESS), DEAD_SHARES,
        ↳ "dead-share lock replication");
}

/**
 * @notice A) Official SnUSD shape: with maxLPTYieldShareWAD == 0 the LPT
 * ↳ premium NEVER accrues – before or after
 *      a full real-holder exit – so the only stranded claim is the
 * ↳ floor(A*S/(S+1)) dust.
 */
function test_PremiumDisabledShape_NoPremiumEver_ExitStrandsOnlyDust() public {
    _deployMarket(cellB(), _officialSnUsdShapeParams());

    _seedMarket(1000e6, 500e6); // 1000 ST units, 500 JT units (6-decimal
    ↳ collateral shares)
    _seedLPT(2000e18, 0, 2000e18); // quote-only LPT seed, 18-decimal quote:
    ↳ 2000 NAV
    _lockDeadShares();

    // A full premium window: 1 day elapsed, +100% collateral gain
    _warpAndRefreshFeed(1 days);
    applySTPNL(10_000);
    SyncedAccountingState memory state = _sync();

    emit log_named_uint("A_premium_accrued_nav",
        ↳ toUint256(state.lptLiquidityPremium));
    emit log_named_uint("A_idle_premium_st_shares", _lptIdlePremiumStShares());
    assertEq(toUint256(state.lptLiquidityPremium), 0, "SnUSD shape must accrue
        ↳ zero LPT premium");
    assertEq(_lptIdlePremiumStShares(), 0, "SnUSD shape must hold zero idle
        ↳ premium ST shares");

    // A full real-holder exit through the multi-asset path
    uint256 claimBefore = _lptClaimNav();
    uint256 realShares = liquidityProviderTranche.balanceOf(LPT_PROVIDER);
    uint256 supplyBefore = liquidityProviderTranche.totalSupply();
    vm.prank(LPT_PROVIDER);
    liquidityProviderTranche.redeemMultiAsset(realShares, 0, 0, LPT_PROVIDER,
        ↳ LPT_PROVIDER);
```



```
uint256 residualNav = _lptClaimNav();
emit log_named_uint("A_claim_before_exit_nav", claimBefore);
emit log_named_uint("A_supply_after_exit",
    ↳ liquidityProviderTranche.totalSupply());
emit log_named_uint("A_residual_nav", residualNav);
assertEq(liquidityProviderTranche.totalSupply(), DEAD_SHARES, "full exit
    ↳ must leave only dead shares");
// The stranded claim is the dead+virtual slice: claim * (DEAD+1)/(supply+1)
    ↳ - dust at any healthy seed size
uint256 expectedResidual = claimBefore.mulDiv(DEAD_SHARES + 1, supplyBefore
    ↳ + 1, Math.Rounding.Ceil) + WAD; // +1 whole unit tolerance for venue
    ↳ rounding
assertLe(residualNav, expectedResidual, "SnUSD-shape residual must stay
    ↳ dust-scale");

// Post-exit gain: still zero premium accrues to the dead-only tranche
_warpAndRefreshFeed(1 days);
applySTPNL(5_000);
state = _sync();
emit log_named_uint("A_postexit_premium_accrued_nav",
    ↳ toUint256(state.lptLiquidityPremium));
emit log_named_uint("A_postexit_claim_nav", _lptClaimNav());
assertEq(toUint256(state.lptLiquidityPremium), 0, "SnUSD shape must keep
    ↳ premium at zero post-exit");
assertEq(_lptIdlePremiumStShares(), 0, "no premium may materialize on the
    ↳ dead-only tranche");
}

/**
 * @notice B) Premium-enabled shape: ordinary redemptions compress the real LPT
    ↳ supply to zero (supply ==
 *     DEAD_SHARES) while the tranche still earns the senior-yield liquidity
    ↳ premium. After the exit, 100% of
 *     every further premium accrues to the dead-only tranche – value
    ↳ reassigned out of senior yield into an
 *     address that can never act. A later depositor cannot capture the
    ↳ locked pile: they mint at fair pricing
 *     against totalValue (which includes the locked claim), and a full exit
    ↳ returns only their own deposit.
 */
function test_PremiumEnabled_FullExit_PremiumLocksToDead_NoCapture() public {
    _deployMarket(cellB(), _premiumEnabledParams());

    _seedMarket(1000e6, 500e6);
    _seedLPT(2000e18, 0, 2000e18);
    _lockDeadShares();

    // Premium cycle while real holders exist: 1 day, +100% gain, 31% of the
    ↳ senior gain to the LPT
    _warpAndRefreshFeed(1 days);
    applySTPNL(10_000);
    SyncedAccountingState memory state = _sync();
    emit log_named_uint("B_preexit_premium_accrued_nav",
        ↳ toUint256(state.lptLiquidityPremium));
    emit log_named_uint("B_preexit_idle_premium_st_shares",
        ↳ _lptIdlePremiumStShares());
    assertGt(toUint256(state.lptLiquidityPremium), 0, "premium-enabled shape
        ↳ must accrue an LPT premium");
    assertGt(_lptIdlePremiumStShares(), 0, "the premium must materialize as
        ↳ kernel-held ST shares");

    // Full real-holder exit – an ordinary redeemMultiAsset by the only real LP
    uint256 realShares = liquidityProviderTranche.balanceOf(LPT_PROVIDER);
    vm.prank(LPT_PROVIDER);
```




```
liquidityProviderTranche.redeemMultiAsset(realShares, 0, 0, LPT_PROVIDER,
    ↳ LPT_PROVIDER);
assertEq(liquidityProviderTranche.totalSupply(), DEAD_SHARES, "REACHABLE:
    ↳ supply compressed to dead shares only");
emit log_named_uint("B_postexit_supply",
    ↳ liquidityProviderTranche.totalSupply());
emit log_named_uint("B_postexit_claim_nav", _lptClaimNav());

// The pathological state persists and keeps earning: another gain pays the
    ↳ premium to the dead-only tranche
_warpAndRefreshFeed(1 days);
applySTPNL(5_000);
state = _sync();
uint256 lockedPremiumNav = toUint256(state.lptLiquidityPremium);
emit log_named_uint("B_postexit_premium_accrued_nav", lockedPremiumNav);
emit log_named_uint("B_postexit_idle_premium_st_shares",
    ↳ _lptIdlePremiumStShares());
assertGt(lockedPremiumNav, 0, "REACHABLE: nonzero senior yield keeps flowing
    ↳ to the dead-only tranche");
assertGt(_lptIdlePremiumStShares(), 0, "the post-exit premium must
    ↳ materialize as ST shares owned by dead supply");
uint256 lockedClaimNav = _lptClaimNav();
emit log_named_uint("B_locked_claim_nav", lockedClaimNav);

// No real holder exists to redeem: the dead address owns 100% of the supply
    ↳ and can never act
assertEq(liquidityProviderTranche.balanceOf(DEAD_ADDRESS), DEAD_SHARES, "the
    ↳ whole supply sits at the dead address");

// A later depositor cannot capture the locked pile: they mint at fair
    ↳ pricing against totalValue (which
// includes the locked claim), and a full exit returns only their own
    ↳ deposit
address lateLp = _generateActor("LATE_LP", LPT_LP_ROLE);
uint256 depositQuote = 1000e18;
quoteToken.mint(lateLp, depositQuote);
vm.startPrank(lateLp);
quoteToken.approve(address(liquidityProviderTranche), depositQuote);
(uint256 mintedShares,) = liquidityProviderTranche.depositMultiAsset(0,
    ↳ depositQuote, 0, lateLp);
(AssetClaims memory stClaims, uint256 quoteOut) =
    liquidityProviderTranche.redeemMultiAsset(mintedShares, 0, 0, lateLp,
        ↳ lateLp);
vm.stopPrank();

uint256 lateLpOutNav = toUint256(stClaims.nav) + quoteOut; // 18-decimal
    ↳ quote is 1:1 with NAV
emit log_named_uint("B_late_lp_minted_shares", mintedShares);
emit log_named_uint("B_late_lp_out_nav", lateLpOutNav);
emit log_named_uint("B_late_lp_deposit_nav", depositQuote);
assertLe(lateLpOutNav, depositQuote, "a later depositor must NOT capture any
    ↳ of the locked residual");
assertGe(lateLpOutNav, depositQuote - 1e12, "a later depositor should
    ↳ recover their deposit modulo rounding dust");

// After the late LP exits, the locked claim is still entirely there, still
    ↳ owned only by dead shares
assertEq(liquidityProviderTranche.totalSupply(), DEAD_SHARES, "supply must
    ↳ return to dead-only after the late exit");
uint256 claimAfterLateCycle = _lptClaimNav();
emit log_named_uint("B_locked_claim_after_late_cycle_nav",
    ↳ claimAfterLateCycle);
assertGe(claimAfterLateCycle, lockedClaimNav - 1e12, "the locked claim
    ↳ survives a full later deposit/exit cycle");
```




```
}

/**
 * @notice C) Amplifier: a dust-scale real supply (2e12 real shares vs 1e12
 *   ↳ dead) plus one 31% premium cycle
 *   ↳ inflates the tranche claim to whole tokens; the terminal exit then
 *   ↳ strands the dead shares' pro-rata
 *   ↳ fraction (~1/3 of the claim) – whole tokens, permanently ownerless,
 *   ↳ in a single cycle.
 */
function test_TinyRealSupply_PremiumInflation_StrandsWholeTokens() public {
    _deployMarket(cellB(), _premiumEnabledParams());

    _seedMarket(1000e6, 500e6);
    // Dust-scale LPT seed: 3e12 quote wei (18-decimal quote) => ~3e12 BPT =>
    //   ↳ ~3e12 LPT shares at bootstrap
    _seedLPT(3e12, 0, 3e12);
    _lockDeadShares(); // real supply: ~2e12, dead: 1e12
    uint256 realShares = liquidityProviderTranche.balanceOf(LPT_PROVIDER);
    emit log_named_uint("C_real_shares", realShares);

    // One premium cycle: 1 day, +100% gain => ~310 NAV of premium lands on a
    //   ↳ ~3e12-supply tranche
    _warpAndRefreshFeed(1 days);
    applySTPNL(10_000);
    _sync();
    uint256 claimBefore = _lptClaimNav();
    uint256 supplyBefore = liquidityProviderTranche.totalSupply();
    emit log_named_uint("C_claim_before_exit_nav", claimBefore);
    emit log_named_uint("C_supply_before_exit", supplyBefore);
    assertGt(claimBefore, 100e18, "the premium must inflate the dust-seeded
    ↳ tranche claim to whole tokens");

    // Terminal exit by the only real holder: payout is floor-scaled, the
    //   ↳ dead+virtual fraction stays
    vm.prank(LPT_PROVIDER);
    (AssetClaims memory stClaims, uint256 quoteOut) =
        liquidityProviderTranche.redeemMultiAsset(realShares, 0, 0,
        ↳ LPT_PROVIDER, LPT_PROVIDER);
    uint256 payoutNav = toUint256(stClaims.nav) + quoteOut;

    uint256 residualNav = _lptClaimNav();
    emit log_named_uint("C_terminal_payout_nav", payoutNav);
    emit log_named_uint("C_terminal_residual_nav", residualNav);
    emit log_named_uint("C_terminal_residual_whole_tokens", residualNav / 1e18);
    assertEq(liquidityProviderTranche.totalSupply(), DEAD_SHARES, "terminal exit
    ↳ must leave only dead shares");
    // The dead fraction of a claim this size is whole-token material: claim *
    //   ↳ (DEAD+1)/(supply+1) ~= claim/3
    assertGt(residualNav, 50e18, "the stranded dead-owned claim must be
    ↳ whole-token material");
    assertLe(payoutNav + residualNav, claimBefore + WAD, "payout + residual must
    ↳ reconcile with the pre-exit claim");

    // The residual is permanently ownerless: the dead address cannot redeem,
    //   ↳ and no other path moves it
    assertEq(liquidityProviderTranche.balanceOf(DEAD_ADDRESS), DEAD_SHARES,
    ↳ "only dead shares remain");
    _warpAndRefreshFeed(1 days);
    _sync();
    assertGe(_lptClaimNav(), residualNav - 1e12, "a flat sync must not release
    ↳ the stranded claim");
    emit log_named_uint("C_residual_after_flat_sync_nav", _lptClaimNav());
}
```



```
}
```

Recommendation: Stop paying the liquidity premium to a tranche with no real holders. Two minimal options:

1. **Gate the carve-out on real supply** — in the accountant's premium computation (`RoycoDayAccountant.sol` , where `lptLiquidityPremium` is derived from `stGain`), treat the LPT yield share as zero whenever `liquidityProviderTranche.totalSupply() <= DEAD_SHARES` (the genesis dead shares can never provide liquidity). The senior gain then stays with senior holders instead of being minted to `0x...dEaD` . The dead-share constant is already available in the deployment template and can be mirrored as an immutable on the tranche or kernel.
2. **Attribute the premium only to non-dead supply** — if the premium must keep accruing for accounting continuity, scale it by `(totalSupply - DEAD_SHARES) / totalSupply` and route the dead fraction back into senior NAV (or to the protocol fee recipient), so no value ever accumulates against shares that cannot act.

In either case, add a regression invariant: after any sync in which the real LPT supply is zero, the LPT claim attributable to `DEAD_ADDRESS` -owned shares must not increase, and `lptOwnedSeniorTrancheShares` minted against a dead-only tranche must be zero.

Royco: Acknowledged. The genesis depositor provided real capital and the dead shares represent that genesis contribution, which continues to receive the liquidity premium by design.

Cantina Managed: Acknowledged.

3.2.4 OPTIMIZED / MAX LPT redemptions resolve on gross queued shares, settle on net

Severity: Informational.

Context: (No context files were provided by the reviewer).

Submitted by [NeuroSpire30](#).

`OPTIMIZED` and `MAX` LPT redemptions resolve route and capacity on the **gross** queued share count **before** queued-yield forfeiture, while settlement only redeems the **post-forfeiture net** share count. When the market appreciates while the request is queued (forfeiture `f > 0`), the gross request can exceed the conservative redemption bounds even though the user's fully-entitled net redemption fits entirely in-kind from the same accounting state — the maximizer then either partial-fills (leaving a spurious remainder queued) or forces a needless multi-asset venue exit, or both.

$$\text{net}(R) \leq \text{maxRedeemInKind} < R$$

$\downarrow$ the user's full entitlement fits the in-kind bound
 $\downarrow$ what the resolution step sees and rejects (the gross request)

- **What breaks:** the exit-route resolution compares the **gross** request `R` against `maxRedeem / maxRedeemMultiAsset` , but settlement forfeits first and only serves `net(R) = R - protocolFeeShares` .
- **When it fires:** any queued LPT redemption where the market appreciated during the queue (`f > 0`) and the in-kind bound binds below the gross request.
- **Who is affected:** every `OPTIMIZED` or `MAX` (sentinel) redemption — the primary automated exit flows — in the profitable-while-queued case, i.e. exactly when users most want out.
- **Same-state proof:** an explicit `INKIND` full-amount redemption of the identical request, executed from the same synced state, clears the whole request in-kind.

Queued redemption requests escrow shares at request time. To keep the user's exit value honest, any value the escrow **accrues while queued** is skimmed as protocol fees at execution (`_redeemWithValueForfeiture`): the escrow's execution-time value is measured against the request-time reference and the delta is forfeited. The user is guaranteed at least their request-time value; the protocol captures the queue-time yield.



The bug is that the *route/capacity resolution* runs **before** this forfeiture is computed, on the **gross** share count — while *settlement* only redeems the **net** share count.

The bug:

Step 1: resolve route/capacity on the GROSS request: `executeRedemption` resolves the fill amount and the exit route before any forfeiture math (`src/entrypoint/RoycoDayEntryPoint.sol` :

```
// Resolve the actual amount of shares to redeem and the exit route from the
↳ request's redemption mode
bool executeMultiAssetRedemption;
if (request.mode == RedemptionMode.OPTIMIZED) {
    (_sharesToRedeem, executeMultiAssetRedemption) =
        _resolveOptimizedRedemption(tranche, (_sharesToRedeem == type(uint256).max)
        ↳ ? request.shares : _sharesToRedeem);
} else {
    executeMultiAssetRedemption = (request.mode == RedemptionMode.MULTIASSET);
    if (_sharesToRedeem == type(uint256).max) {
        _sharesToRedeem = Math.min(
            (executeMultiAssetRedemption ? _maxRedeemMultiAsset(tranche) :
            ↳ IRoycoVaultTranche(tranche).maxRedeem(address(this))),
            ↳ request.shares
        );
    }
}
```

`_resolveOptimizedRedemption` compares the bounds against the **gross target**:

```
// In-kind whenever it can serve the entire target
uint256 maxRedeemInKind = IRoycoVaultTranche(_tranche).maxRedeem(address(this));
if (maxRedeemInKind >= _targetSharesToRedeem) return (_targetSharesToRedeem, false);
// Otherwise fill up to the dominant bound, exiting multi-asset only when its bound
↳ is strictly wider
uint256 maxRedeemMultiAsset = _maxRedeemMultiAsset(_tranche);
return (Math.min(Math.max(maxRedeemInKind, maxRedeemMultiAsset),
↳ _targetSharesToRedeem), maxRedeemMultiAsset > maxRedeemInKind);
```

The bounds themselves are **view bounds over the full escrow state** — they have no knowledge of the forfeiture the same execution is about to apply:

```
// RoycoVaultTranche.maxRedeem - L114
function maxRedeem(address _owner) public view virtual override(IRoycoVaultTranche)
↳ returns (uint256 shares) {
    return Math.min(balanceOf(_owner),
        ↳ IRoycoDayKernel(kernel()).inkindMaxRedeemable(_owner));
}

// RoycoLiquidityProviderTranche.maxRedeemMultiAsset - L128
function maxRedeemMultiAsset(address _owner) external virtual
↳ override(IRoycoLiquidityProviderTranche) returns (uint256 shares) {
    return Math.min(balanceOf(_owner),
        ↳ IRoycoDayKernel(kernel()).lptMaxRedeemableMultiAsset(_owner));
}
```

Step 2: forfeit, then redeem only the NET slice: Only afterwards does `_redeemWithValueForfeiture` compute the forfeiture and redeem the net remainder (`src/entrypoint/RoycoDayEntryPoint.sol` :

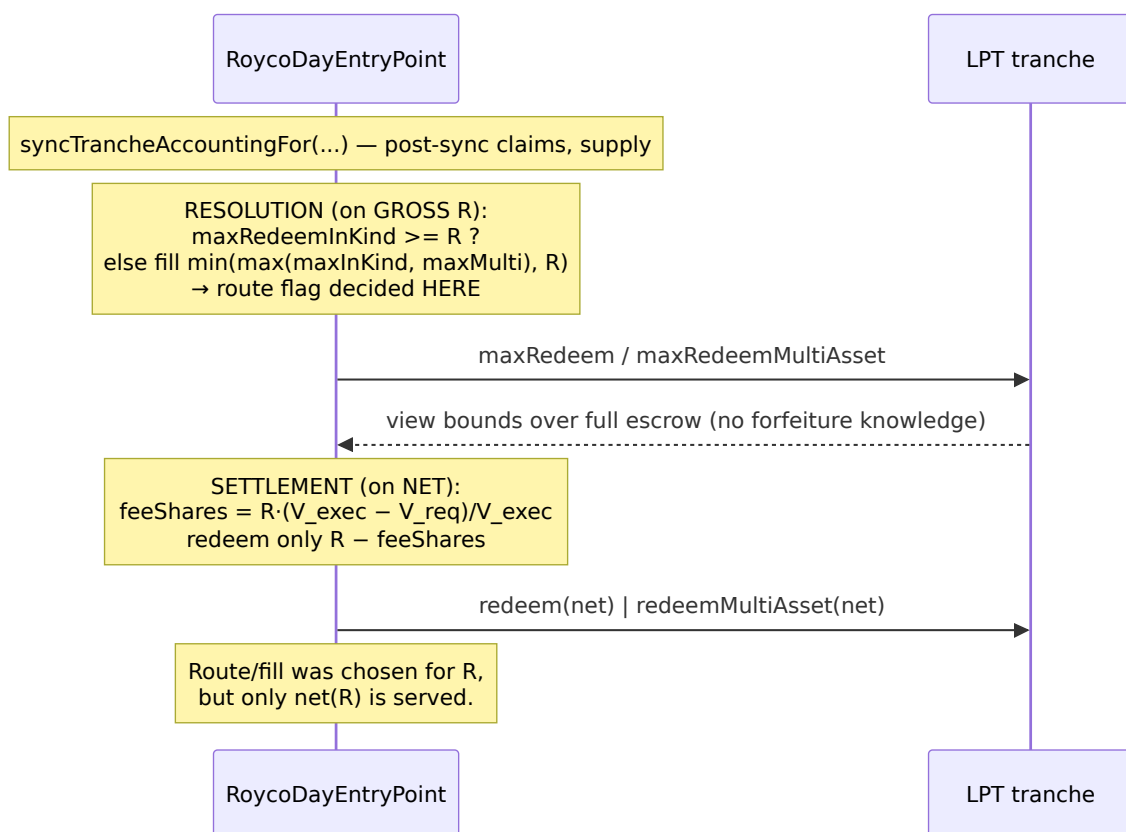


```

userSharesRedeemed = _shares;
// Compute the value of the shares at execution against the caller's synced claims,
// without any self-liquidation bonus applied
NAV_UNIT valueAtExecutionTime = _redemptionValueReference(_shares, _trancheClaims,
    ↪ _totalTrancheShares);
if (valueAtExecutionTime > _valueAtRequestTime) {
    protocolFeeShares = _shares.mulDiv((valueAtExecutionTime -
    ↪ _valueAtRequestTime), valueAtExecutionTime, Math.Rounding.Floor);
}
// Redeem the shares the user is entitled to after deducting the protocol fee shares
// A fully forfeited redemption (a zero-value snapshot) settles without a redeem
// call, mirroring the deposit path
if ((userSharesRedeemed -= protocolFeeShares) != 0) {
    if (_executeMultiAssetRedemption) {
        (userClaims, quoteAssets) = IRoycoLiquidityProviderTranche(_tranche).redeem(
        ↪ MultiAsset(userSharesRedeemed, 0, 0, _receiver,
        ↪ address(this)));
    } else {
        userClaims = IRoycoVaultTranche(_tranche).redeem(userSharesRedeemed,
        ↪ _receiver, address(this));
    }
}
}

```

Step 3: the mismatch:



The resolution step treats the whole gross request as if it had to be served. But settlement only serves `net(R) = R - feeShares(R)`. The resolution never sees the forfeiture, so a request whose net entitlement fits the in-kind bound is rejected by the gross comparison and under-served.

Root cause: Ordering. In `executeRedemption`, route/capacity resolution consumes the **gross** `_sharesToRedeem`, while `_redeemWithValueForfeiture` redeems only `_shares - protocolFee`.



Shares . The two steps disagree on how many shares actually need to be served — by exactly the forfeiture amount.

Because the forfeiture ratio $\text{feeShares}(R)/R$ is **slice-invariant** (both the request-time reference and the execution-time value reference scale linearly in shares), $\text{net}(R)$ is *exactly* the entitlement an explicit full redemption settles — so the state genuinely can serve the whole request, and gross-based resolution is strictly worse than the explicit path on the same state.

Trigger conditions:

#	Condition	Notes
1	Queued LPT redemption (any mode)	The standard exit flow
2	Market appreciates while queued $\rightarrow \text{feeShare}_s > 0$	Normal, profitable operation — forfeiture is largest exactly when the market did best
3	$\text{maxRedeemInKind} < R$	The in-kind bound binds below the gross request — routine for large positions (the bound applies the liquidity-requirement haircut and excludes the idle premium the in-kind exit pays out)
4	$\text{net}(R) \leq \text{maxRedeemInKind}$	The user's full entitlement still fits in-kind — the band that makes the under-serve <i>artificial</i>

The missing quadrant the proof of concept covers — $\text{net}(\text{full request}) \leq \text{maxRedeemInKind} < \text{gross request}$ — is the intersection of "the market performed well" with "a conservative in-kind bound": a routine state, not an edge case.

Proof of concept:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import { Math } from
  ↳ "../.../lib/openzeppelin-contracts/contracts/utils/math/Math.sol";
import { IRoycoDayEntryPoint } from
  ↳ "../.../src/interfaces/IRoycoDayEntryPoint.sol";
import { AssetClaims } from "../.../src/libraries/Types.sol";
import { toUint256 } from "../.../src/libraries/Units.sol";
import { EntryPointTestBase } from "../.../utils/EntryPointTestBase.sol";
import { defaultParams } from "../.../utils/MarketParams.sol";
import { cellA } from "../.../utils/TokenConfigs.sol";

/**
 * @title Test_PoC_OptimizedResolvesGrossNotNet
 * @notice PoC for: OPTIMIZED / MAX LPT redemptions resolve route and capacity on
 * ↳ the GROSS queued share count
 * ↳ BEFORE queued-yield forfeiture, while settlement only redeems the
 * ↳ POST-forfeiture NET share count.
 * ↳ When the global bound B satisfies  $\text{net}(R) \leq B < R$ , the whole request IS
 * ↳ executable in one shot on a
 * ↳ net basis from the same state, but the maximizer cannot see it: it
 * ↳ partial-fills (and/or misroutes
 * ↳ multi-asset) leaving a spurious remainder queued.
 * ↳
 * ↳ Fixture: the LPT routing wedge (senior-share-heavy pool leg so the
 * ↳ in-kind bound binds below the
 * ↳ escrow) + a staged idle liquidity premium + a BPT gain during the queue
 * ↳ so the LT share underperforms
```



```
*         and forfeiture f > 0. The premium is staged BEFORE the request so it is
↳ in the request-time reference
*         and the BPT gain lands purely as queue-time accrual.
*/
contract Test_PoC_OptimizedResolvesGrossNotNet is EntryPointTestBase {
    using Math for uint256;

    /// @dev One whole quote token in its native decimals (this market's quote asset
    ↳ uses 6 decimals, so 1e6)
    uint256 internal QUOTE_UNIT;

    function setUp() public {
        _deployMarket(cellA(), defaultParams());
        QUOTE_UNIT = 10 ** uint256(cell.quoteAsset.decimals);
        _seedMarket(100_000e18, 30_000e18);
        _seedLPT(10_000e18, 8000e18, 2000 * QUOTE_UNIT);
        _deployEntryPoint();
    }

    /// @dev Stages an idle liquidity premium: armed venue slippage makes the
    ↳ premium's reinvest gate fail, so the
    /// kernel holds the premium as idle senior tranche shares that LPT
    ↳ redemptions pay out in kind
    function _stageIdlePremium() internal {
        setVenueSlippageMode(true);
        applySTPNL(1000);
        _sync();
        assertGt(kernel.getState().lptOwnedSeniorTrancheShares, 0, "the fixture must
        ↳ stage an idle senior-share premium");
    }

    /// @dev Hands the whole seeded LPT position to USER_A, stages the premium, and
    ↳ queues USER_A's whole position
    /// under the given mode, then accrues a BPT gain during the queue
    ↳ (forfeiture > 0) and warps past the delay
    function _arrangeQueuedGain(
        IRoycoDayEntryPoint.RedemptionMode _mode
    )
    internal
    returns (uint256 nonce, uint256 escrowedShares, uint256 feeSharesBefore)
    {
        // The premium must be staged before the request so it sits in the
        ↳ request-time value reference
        _stageIdlePremium();

        // Hand the seeded LPT position to USER_A so the escrow can cross the
        ↳ in-kind bound
        uint256 seededShares = liquidityProviderTranche.balanceOf(LPT_PROVIDER);
        vm.prank(LPT_PROVIDER);
        liquidityProviderTranche.transfer(USER_A, seededShares);

        (nonce,) = _requestRedemption(USER_A, address(liquidityProviderTranche),
        ↳ seededShares, USER_A, 0, _mode);
        escrowedShares = entryPoint.getRedemptionRequest(USER_A, nonce).shares;
        feeSharesBefore = entryPoint.getProtocolFeeSharesPendingCollection(address(
        ↳ liquidityProviderTranche));

        // BPT appreciates 100% while queued: the staged idle premium makes the LT
        ↳ share underperform the BPT gain,
        // so the queue-time accrual registers as forfeitable yield (f > 0)
        applyLPTPNL(10000);
        _warpPastRedemptionDelay();
    }
}
```



```
/// @dev Derives the net share count a GROSS execution of _gross shares settles
→ to, from the fee fraction an
/// already-settled slice of the same state booked (the forfeiture ratio is
→ slice-invariant: both the
/// request-time reference and the execution-time value reference scale
→ linearly in shares)
function _netOf(uint256 _gross, uint256 _sliceFeeShares, uint256 _sliceGross)
→ internal pure returns (uint256) {
    return _gross - Math.mulDiv(_gross, _sliceFeeShares, _sliceGross,
    → Math.Rounding.Floor);
}

/// @notice OPTIMIZED + MAX: with net(R) <= maxRedeemInKind < R the maximizer
→ still resolves gross (partial
/// fill) and routes multi-asset, leaving a spurious remainder – even
→ though the whole request was
/// fully executable in-kind on a net basis from this exact state
function test_poc_optimizedMax_grossResolution_partialFillsAndMisroutes()
→ public {
    (uint256 nonce, uint256 R, uint256 feeSharesBefore) =
        _arrangeQueuedGain(IRoycoDayEntryPoint.RedemptionMode.OPTIMIZED);

    // Post-queue, post-sync bounds against the entry point's escrow
    uint256 maxInKind = liquidityProviderTranche.maxRedeem(address(entryPoint));
    uint256 maxMulti =
        → liquidityProviderTranche.maxRedeemMultiAsset(address(entryPoint));

    emit log_named_uint("R (gross queued shares)", R);
    emit log_named_uint("maxRedeemInKind", maxInKind);
    emit log_named_uint("maxRedeemMultiAsset", maxMulti);

    // Execute the maximizer
    (AssetClaims memory claims, uint256 quoteAssets) =
        → _executeRedemptionMaxWithQuote(USER_A, USER_A, nonce);
    uint256 remainder = entryPoint.getRedemptionRequest(USER_A, nonce).shares;
    uint256 grossResolved = R - remainder;
    uint256 feeShares = entryPoint.getProtocolFeeSharesPendingCollection(address
    → s(liquidityProviderTranche)) -
        → feeSharesBefore;

    emit log_named_uint("grossResolved (gross slice consumed)", grossResolved);
    emit log_named_uint("netRedeemed (post-forfeiture)", grossResolved -
        → feeShares);
    emit log_named_uint("protocolFeeShares", feeShares);
    emit log_named_uint("remainderQueued", remainder);
    emit log_named_uint("quoteAssets (multi-asset leg)", quoteAssets);

    // The band premise: the full request's NET entitlement fits the in-kind
    → bound, the gross does not
    uint256 netOfFull = _netOf(R, feeShares, grossResolved);
    emit log_named_uint("netOfFull (full request net entitlement)", netOfFull);
    assertLt(maxInKind, R, "premise: the in-kind bound must bind below the gross
    → request");
    assertLe(netOfFull, maxInKind, "premise: the full request's net entitlement
    → must fit the in-kind bound");

    // The bug: gross-based resolution under-delivers and misroutes despite full
    → net serviceability
    assertGt(remainder, 0, "BUG: the maximizer must leave a spurious remainder
    → it could have served net");
    assertGt(grossResolved, maxInKind, "the gross slice consumed crosses the
    → in-kind bound");
    assertGt(quoteAssets, 0, "BUG: the maximizer must route multi-asset although
    → in-kind could serve the net");
```




```

    assertEq(toUint256(claims.lptAssets), 0, "a multi-asset exit carries no
    ↪ LP-token leg");
}

// @notice Control: the SAME state and request, executed as an explicit INKIND
↪ full-amount redemption, fully
// ↪ clears the request in-kind on the net basis – proving the
↪ maximizer's partial fill was artificial
function test_poc_explicitInKindFullAmount_clearsWholeRequest() public {
    (uint256 nonce, uint256 R,) =
    ↪ _arrangeQueuedGain(IRoycoDayEntryPoint.RedemptionMode.INKIND);

    uint256 maxInKind = liquidityProviderTranche.maxRedeem(address(entryPoint));
    emit log_named_uint("R (gross queued shares)", R);
    emit log_named_uint("maxRedeemInKind", maxInKind);
    emit log_named_uint("maxRedeemMultiAsset",
    ↪ liquidityProviderTranche.maxRedeemMultiAsset(address(entryPoint)));

    // The explicit INKIND full-amount path passes the GROSS request straight to
    ↪ _redeemWithValueForfeiture,
    // ↪ which forfeits first and only then redeems the NET slice the kernel must
    ↪ actually serve
    (AssetClaims memory claims, uint256 quoteAssets) =
    ↪ _executeRedemptionWithQuote(USER_A, USER_A, nonce, R);

    uint256 remainder = entryPoint.getRedemptionRequest(USER_A, nonce).shares;
    emit log_named_uint("remainderQueued", remainder);
    emit log_named_uint("quoteAssets (multi-asset leg)", quoteAssets);

    assertEq(remainder, 0, "control: the whole request must clear on the net
    ↪ basis");
    assertEq(quoteAssets, 0, "control: the exit must be in-kind");
    assertGt(toUint256(claims.lptAssets), 0, "control: the in-kind exit must pay
    ↪ the LP-token leg");
    assertGt(claims.stShares, 0, "control: the in-kind exit must pay the idle
    ↪ premium senior-share leg");
}

// @dev Executes a redemption request as _executor for the specified shares,
↪ returning the quote leg alongside the claims
function _executeRedemptionWithQuote(
    address _executor,
    address _user,
    uint256 _nonce,
    uint256 _shares
)
    internal
    returns (AssetClaims memory claims, uint256 quoteAssets)
{
    vm.prank(_executor);
    (claims, quoteAssets) = entryPoint.executeRedemption(_user, _nonce,
    ↪ _shares);
}
}

```

test/concrete/EntryPoint/Test_PoC_OptimizedResolvesGrossNotNet.t.sol — 2/2 pass:

```

[PASS] test_poc_optimizedMax_grossResolution_partialFillsAndMisroutes() (gas:
↪ 2867596)
[PASS] test_poc_explicitInKindFullAmount_clearsWholeRequest() (gas: 2060395)

```

Fixture: Mirrors the existing multi-asset routing wedge, with the idle premium staged **before** the request so the BPT gain lands purely as queue-time accrual:



```

function setUp() public {
    _deployMarket(cellA(), defaultParams());
    QUOTE_UNIT = 10 ** uint256(cell.quoteAsset.decimals);
    _seedMarket(100_000e18, 30_000e18);
    _seedLPT(10_000e18, 8000e18, 2000 * QUOTE_UNIT); // senior-heavy leg → in-kind
    → bound binds
    _deployEntryPoint();
}

function _arrangeQueuedGain(RedemptionMode _mode) internal returns (uint256 nonce,
    → uint256 escrowedShares, uint256 feeSharesBefore) {
    _stageIdlePremium(); // idle premium staged BEFORE
    → the request
    uint256 seededShares = liquidityProviderTranche.balanceOf(LPT_PROVIDER);
    vm.prank(LPT_PROVIDER);
    liquidityProviderTranche.transfer(USER_A, seededShares);
    (nonce,) = _requestRedemption(USER_A, address(liquidityProviderTranche),
    → seededShares, USER_A, 0, _mode);
    escrowedShares = entryPoint.getRedemptionRequest(USER_A, nonce).shares;
    feeSharesBefore = entryPoint.getProtocolFeeSharesPendingCollection(address(liqui
    → dityProviderTranche));
    applyLPTpNL(10000); // BPT +100% while queued →
    → forfeiture > 0
    _warpPastRedemptionDelay();
}

```

Bug test: OPTIMIZED + MAX sentinel: Resolution on gross, settlement on net, from the band $\text{net}(R) \leq \text{maxInKind} < R$:

Metric	Value	Meaning
R (gross queued)	15.401e21	the request the resolution sees
maxRedeemInKind	11.547e21	in-kind bound
maxRedeemMultiAsset	11.787e21	multi-asset bound
gross resolved (consumed)	11.787e21	$\min(\max(11.547, 11.787), R) \rightarrow$ multi-asset route
net redeemed (post-forfeiture)	8.194e21	what is actually served
protocol fee shares	3.593e21	forfeited queue-time accrual
remainder queued	3.614e21	spurious partial fill
quote leg	3,937,470,553	spurious multi-asset / quote-form exit
net(R) (full entitlement)	10.706e21	$\leq \text{maxRedeemInKind}$ — would have cleared in-kind

```

assertGt(remainder, 0, "BUG: the maximizer must leave a spurious remainder it could
    → have served net");
assertGt(grossResolved, maxInKind, "the gross slice consumed crosses the in-kind
    → bound");
assertGt(quoteAssets, 0, "BUG: the maximizer must route multi-asset although in-kind
    → could serve the net");

```

Control test — explicit INKIND full amount, same state:

```

vm.prank(USER_A);
(claims, quoteAssets) = entryPoint.executeRedemption(USER_A, nonce, R);

```



```
remainder = 0    quoteAssets = 0    lptAssets > 0    stShares > 0
```

The identical request clears **entirely in-kind**. The partial fill and the misroute are pure artifacts of gross-based resolution.

Impact: The two artifacts, in order of size:

1. partial fill — the dominant harm. The maximizer caps below the gross request, **leaving a spurious remainder queued that requires another execution attempt**. Consequences:

- **Capital lockup and continued market risk** on the remainder: the user wanted out now and is forced to stay exposed (forfeiture only skims *appreciation*; on depreciation the remainder settles at the lower, current NAV).
- **Window expiry:** in fixed-term markets with long redemption windows, the window can lapse between attempts — **the user may then be forced to cancel instead of fully exiting**, or to wait for the next window.
- **Repeated executor bonuses** if third-party execution is used (each attempt pays the bonus on its slice).
- No compounding forfeiture loss: the fee ratio is slice-invariant, so a re-attempt forfeits proportionally the same — the user is never underpaid versus their request-time entitlement. This bounds the impact.

2. Wrong route selection. When `maxRedeemMultiAsset > maxRedeemInKind`, the exit is routed multi-asset **even though the net slice would have exited entirely in-kind**. This **forces a needless multi-asset venue exit and quote-form settlement** — it **pushes the user onto the venue-exit path and changes the asset basket** (quote + collateral instead of the BPT + idle-premium-ST position). Where the user intended to keep LP exposure, they must re-enter at fresh entry friction (deposit costs, imbalance in real E-CLP venues). Note: the multi-asset removal is *proportional*, so this is **not a swap-fee bleed** — the cost is the asset-basket change and re-entry friction, not venue fees.

No principal theft, no bricked requests (the remainder stays executable and cancellable), no adversarial or grieving lever. The loss is bounded by time value, market risk on the remainder, and basket-change friction.

Severity — Medium:

Dimension	Assessment
Impact	Low–Medium — no principal loss; capital lockup + market risk on the remainder (amplified in fixed-term windows), needless venue-exit / asset-basket change, possible forced cancel on window lapse
Likelihood	Medium — normal profitable operation, no attacker, largest exactly when markets do well
Overall	Medium

Why not High: no principal loss, no grieving, no systemic risk, bounds stay conservative, and an explicit `INKIND` / `MULTIASSET` amount dodges it entirely.

Why not Low: the primary automated flows systematically under-serve versus the explicit path on the *same state* — the signature of a bug, not a design choice — and the user cannot predict or control it (mode `OPTIMIZED` delegates the route; the resolution runs before the forfeiture is visible).

Scope / exclusion check — no out-of-scope category applies:

Exclusion category	Does it fall in it?	Why not
--------------------	---------------------	---------



Previous security reports	No	Novelty verified: the only prior finding touching these bounds is S1 (invalidated — different component/root cause) and Certora L-07 (UX notes, no gross/net split). No prior report describes forfeiture-vs-route-resolution ordering.
Expected behaviors / accepted risks / trusted roles	No	Forfeiture itself is documented, but the <i>ordering</i> (resolve on gross, settle on net) is not an accepted risk — it makes the automated path strictly worse than the explicit path on the same state, which no design rationale covers. No role assumptions are involved.
Oracle, NAV, or share price manipulation	No	The bug fires in normal, non-adversarial operation (market appreciation while queued). No attacker lever, no oracle mispricing, no price manipulation — the band arises from honest market moves against conservative bounds.
Whitelisted parties behaving maliciously	No	No trusted party acts maliciously; no privileged role is required to trigger or exploit. Any user queuing an LPT redemption in a profitable-while-queued market is affected.
Incorrect amounts to whitelisted parties / recipients (reversible)	No	Amounts settled are correct per the forfeiture design (user always receives $\geq$ request-time value). The flaw is underserved <i>scope and route</i> , not incorrect payouts, and it is not about whitelisted recipients.
External protocol bugs	No	Entirely within Royco's own code (RoycoDayEntryPoint.sol); the venue (Balancer V3 mock) behaves correctly and proportionally. No external dependency misbehaves.
Centralization risks	No	No admin/owner action, no upgrade involved, no privileged-set dependence.
MEV, gas grieving, frontrunning	No	Not MEV (no ordering-based value extraction; the outcome is deterministic per state), not gas grieving (no grieving attacker; the cost falls on the affected user's own request), not frontrunning (no victim transaction is raced).
Frontend or off-chain components	No	Pure on-chain smart-contract logic in <code>executeRedemption</code> .

Suggested fix: Resolve route and capacity on the **post-forfeiture net** share count. Since the forfeiture ratio is slice-invariant, `net(R)` is exact and computable up front:

```
// At the top of executeRedemption, after the sync and BEFORE resolving
↪ route/capacity:
NAV_UNIT valueAtExecutionTime = _redemptionValueReference(R, trancheClaims,
↪ totalTrancheShares);
uint256 protocolFeeShares = (valueAtExecutionTime > request.valueAtRequestTime)
? R.mulDiv(valueAtExecutionTime - request.valueAtRequestTime,
↪ valueAtExecutionTime, Math.Rounding.Floor)
: 0;
```



```
uint256 netTarget = R - protocolFeeShares; // the slice the bounds must actually
↳ serve

// Resolve route/capacity against netTarget (OPTIMIZED / MAX sentinel), then settle
↳ exactly netTarget
```

Minimal variant: pass the forfeiture-adjusted target into `_resolveOptimizedRedemption` and the sentinel clamp (the ratio being slice-invariant makes this exact), so the resolved fill and the redeemed slice agree — or restructure so the forfeiture is computed once at the top and shared by both steps.

Royco: Acknowledged. The behavior is intended: the redemption is evaluated on the gross request as a whole, regardless of how much the user specifically receives after protocol fee forfeiture.

Cantina Managed: Acknowledged.

3.2.5 Each oracle replacement setter reads the other oracle, one shared outage freezes every operation with no on-chain recovery

Severity: Informational.

Context: (No context files were provided by the reviewer).

Submitted by [heavyw8t](#), also found by [t0x1c](#).

Summary: `RoycoDayKernel.setCollateralAssetOracle` and `BalancerV3LiquidityVenue.setBPTOracle` are the only on-chain paths that replace a failed price oracle. Both end with an unconditional `_preOpSyncTrancheAccountingWithPriceCache()`, and that sync reads both oracles: `withPriceCache` pokes the collateral asset oracle, and the sync values the liquidity provider leg through the BPT oracle.

Each setter therefore depends on the oracle it is not replacing. When one incident takes both unhealthy, the collateral setter reverts on the dead BPT oracle and the BPT setter reverts on the dead collateral oracle. Neither runs, every sync reverts, and the market holds user funds with no on-chain way to restore pricing.

Location: `src/kernels/base/RoycoDayKernel.sol`.

Description: Replacing a broken oracle is the recovery mechanism, so it must not require the broken pricing stack to work. The `withPriceCache` modifier pokes the collateral asset oracle before the body runs, a documented circuit breaker:

```
IRoycoPriceOracle(_getRoycoDayKernelStorage().collateralAssetOracle).poke();
```

`RoycoDayKernel.sol:56-58` . `setCollateralAssetOracle` installs the new oracle then syncs unconditionally at `RoycoDayKernel.sol:364`, and `setBPTOracle` does the same at `BalancerV3LiquidityVenue.sol:309`. The leading sync is optional behind `_syncBeforeUpdate`, the trailing one is not.

[RoycoDayKernel.sol#L358-L365](#).

Steps:

1. One incident makes both oracles report unhealthy.
2. `ADMIN_ORACLE_ROLE` calls `setCollateralAssetOracle` with a healthy replacement. The trailing sync values the liquidity provider leg through the dead BPT oracle and reverts.
3. The same role calls `setBPTOracle` with a healthy replacement. The trailing sync pokes the dead collateral oracle and reverts.
4. `syncTrancheAccounting` reverts likewise, so deposits and redemptions stay shut.

Impact Explanation: Impact: High. Every asset is frozen, 100 percent of senior, junior and liquidity provider deposits, with no on-chain recovery. A market holding 10 million dollars stays shut until a UUPS upgrade ships. With a Low likelihood the cell is Medium.



Likelihood Explanation: Likelihood: Low.

Internal preconditions:

1. None. No role has to misbehave and no attacker is involved.

External preconditions:

1. Both oracles report unhealthy at once. The adapters share a Chainlink dependency and the same sequencer uptime gate, so one upstream incident reaches both.

Proof of Concept: Save as `test/concrete/OracleReplacementDeadlock.t.sol` and run `forge test --match-path test/concrete/OracleReplacementDeadlock.t.sol -vv`. Healthy replacements are minted before the incident, so the only variable is reachability.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.28;

import { MockBPTOracle } from "../mocks/MockBPTOracle.sol";
import { MockPriceOracle } from "../mocks/MockPriceOracle.sol";
import { EntryPointTestBase } from "../utils/EntryPointTestBase.sol";
import { defaultParams } from "../utils/MarketParams.sol";
import { cellA } from "../utils/TokenConfigs.sol";

contract OracleReplacementDeadlockTest is EntryPointTestBase {
    uint256 internal collateralUnit;
    uint256 internal quoteUnit;
    address internal seniorHolder;
    address internal juniorHolder;

    function setUp() public {
        _deployMarket(cellA(), defaultParams());
        collateralUnit = 10 ** uint256(cell.collateralAsset.decimals);
        quoteUnit = 10 ** uint256(cell.quoteAsset.decimals);
        _seedMarket(100 * collateralUnit, 30 * collateralUnit);
        _seedLPT(20e18, 0, 20 * quoteUnit);
        _deployEntryPoint();
        seniorHolder = _generateEntryPointUser("SENIOR_HOLDER");
        juniorHolder = _generateEntryPointUser("JUNIOR_HOLDER");
    }

    function _healthyCollateralOracle() internal returns (MockPriceOracle
    ↪ replacement) {
        replacement = new MockPriceOracle(address(stJtVault), collateralPriceWAD,
        ↪ ORACLE_STALENESS_THRESHOLD_SECONDS);
        replacement.setUpdatedAt(vm.getBlockTimestamp());
    }

    function _healthyBPTOracle(uint256 _tv1) internal returns (MockBPTOracle
    ↪ replacement) {
        replacement = new MockBPTOracle(balancerVault, address(bpt));
        replacement.setTVL(_tv1);
        replacement.setMode(MockBPTOracle.Mode.MANUAL);
    }

    function _openPositions() internal {
        _acquireTrancheShares(seniorHolder, address(seniorTranche), 10 *
        ↪ collateralUnit);
        _acquireTrancheShares(juniorHolder, address(juniorTranche), 10 *
        ↪ collateralUnit);
        kernel.syncTrancheAccounting();
    }

    function test_bothOraclesUnhealthyLockBothReplacementPaths() public {
        _openPositions();
    }
}
```



```
uint256 liveTVL = bptOracle.computeTVL();

// Healthy replacements minted BEFORE the incident, so the only thing under
↳ test is reachability
MockPriceOracle goodCollateral = _healthyCollateralOracle();
MockBPTOracle goodBPT = _healthyBPTOracle(liveTVL);

// One incident takes both oracles down
collateralAssetOracle.setRevertMode(true);
bptOracle.setRevertMode(true);

// The collateral oracle cannot be replaced: the trailing sync reads the
↳ dead BPT oracle
vm.prank(ORACLE_ADMIN);
vm.expectRevert();
kernel.setCollateralAssetOracle(address(goodCollateral), false);

// The BPT oracle cannot be replaced: the trailing sync pokes the dead
↳ collateral oracle
vm.prank(ORACLE_ADMIN);
vm.expectRevert();
kernel.setBPTOracle(address(goodBPT), false);

// The ordinary sync surface is shut too, so the book cannot be advanced by
↳ anyone
vm.expectRevert();
kernel.syncTrancheAccounting();

emit log("both replacement setters and the public sync revert: no on-chain
↳ recovery path");
}

// CONTROL A: only the collateral oracle is down, so its own setter still works
function test_control_collateralOracleReplaceableWhileBPTOracleIsLive() public {
    _openPositions();

    MockPriceOracle goodCollateral = _healthyCollateralOracle();
    collateralAssetOracle.setRevertMode(true);

    vm.prank(ORACLE_ADMIN);
    kernel.setCollateralAssetOracle(address(goodCollateral), false);

    kernel.syncTrancheAccounting();
    emit log("collateral oracle replaced and the market resumed while the BPT
↳ oracle stayed live");
}

// CONTROL B: only the BPT oracle is down, so its own setter still works
function test_control_bptOracleReplaceableWhileCollateralOracleIsLive() public {
    _openPositions();
    uint256 liveTVL = bptOracle.computeTVL();

    MockBPTOracle goodBPT = _healthyBPTOracle(liveTVL);
    bptOracle.setRevertMode(true);

    vm.prank(ORACLE_ADMIN);
    kernel.setBPTOracle(address(goodBPT), false);

    kernel.syncTrancheAccounting();
    emit log("BPT oracle replaced and the market resumed while the collateral
↳ oracle stayed live");
}

function test_deadlockPersistsAcrossTime() public {
```




```

    _openPositions();
    uint256 liveTVL = bptOracle.computeTVL();
    MockPriceOracle goodCollateral = _healthyCollateralOracle();
    MockBPTOracle goodBPT = _healthyBPTOracle(liveTVL);

    collateralAssetOracle.setRevertMode(true);
    bptOracle.setRevertMode(true);

    vm.warp(vm.getBlockTimestamp() + 30 days);

    vm.prank(ORACLE_ADMIN);
    vm.expectRevert();
    kernel.setCollateralAssetOracle(address(goodCollateral), false);

    vm.prank(ORACLE_ADMIN);
    vm.expectRevert();
    kernel.setBPTOracle(address(goodBPT), false);

    emit log("30 days later both replacement paths are still shut");
}
}

```

```

[PASS] test_bothOraclesUnhealthyLockBothReplacementPaths() (gas: 1627824)
  both replacement setters and the public sync revert: no on-chain recovery path
[PASS] test_control_bptOracleReplaceableWhileCollateralOracleIsLive() (gas: 1353685)
  BPT oracle replaced and the market resumed while the collateral oracle stayed live
[PASS] test_control_collateralOracleReplaceableWhileBPTOracleIsLive() (gas: 1029141)
  collateral oracle replaced and the market resumed while the BPT oracle stayed live
[PASS] test_deadlockPersistsAcrossTime() (gas: 1600889)
  30 days later both replacement paths are still shut
Suite result: ok. 4 passed; 0 failed; 0 skipped

```

Both replacement calls revert while both oracles are down. Each control breaks one oracle, replaces it and resumes the market, isolating the cross-dependency.

Recommendation: Remove the trailing `_preOpSyncTrancheAccountingWithPriceCache()` from `setCollateralAssetOracle` and from `setBPTOracle`. The next ordinary sync commits the incoming mark.

Where committing inside the setter is preferred, route it through `_tryExecute` so a reverting sync is tolerated, matching `RoycoDayAccountant.setJuniorTrancheYDM`, which uses that pattern because the setter is the only recovery path from a sync-bricking dependency.

Royco: Fixed in commit [3c93ce8f](#).

Cantina Managed: Acknowledged. The protocol assumes oracles are reliable and correctly configured, making simultaneous failure of both oracles an edge case outside the design envelope. In the event of such a failure, a UUPS upgrade can restore recovery access.

3.2.6 A released fixed term is re-stamped in full by the next downtick, freezing all tranche redemptions indefinitely

Severity: Informational.

Context: (No context files were provided by the reviewer).

Submitted by [heavyw8t](#).

Summary: `RoycoDayAccountant._previewSyncTrancheAccounting` releases an elapsed fixed term through condition 5 of its forced-perpetual test, forcing `PERPETUAL` and clearing `fixedTermEndTimestamp`. The release is one-shot: whichever transaction commits a sync first consumes it.



Once consumed, the market re-enters `FIXED_TERM` on the next non-dust drawdown and stamps a full fresh `block.timestamp + fixedTermDurationSeconds`, not the unserved remainder, guarded only by whether the market was perpetual when the sync started. The design bounds one term, not total frozen time.

Location: `src/accountant/RoycoDayAccountant.sol`.

Description: `FIXED_TERM` is a transient junior observation period entered on a drawdown above `dustTolerance`, holding redemptions for `fixedTermDurationSeconds`. The re-entry stamp carries one state guard:

```
if (initialMarketState == MarketState.PERPETUAL) fixedTermEndTimestamp =  
    ↪ uint32(block.timestamp + fixedTermDurationSeconds);
```

`RoycoDayAccountant.sol:561`. Releasing condition 5 sits at `RoycoDayAccountant.sol:546-547`.
[RoycoDayAccountant.sol#L555-L562](#).

No holder is guaranteed to reach it. `DepositLogic.sol:63` exempts liquidity provider deposits from the gate, so an ordinary provider consumes it for everyone, `syncTrancheAccounting` is `PUBLIC_ROLE`, and the entry point commits a sync at four sites.

Steps:

1. A drawdown above `dustTolerance` puts the market in `FIXED_TERM` ending at T.
2. At T an ordinary liquidity provider deposits, taking condition 5, forcing `PERPETUAL` and consuming the release.
3. Twelve seconds later the mark ticks down one basis point.
4. The next sync re-enters `FIXED_TERM` with a full fresh term. All three tranches revert `DISABLED_IN_FIXED_TERM_STATE` and `maxRedeem` returns zero.
5. Repeat. Queued entry-point requests expire on their own clock throughout.

Impact Explanation: Impact: Medium. Every holder in all three tranches loses redemption access for an unbounded time. Measured below: ten terms shut the market for 140 days on a 50 basis point decline. A junior holder with 50,000 dollars cannot exit while the mark slides. With a High likelihood the cell is Medium.

Likelihood Explanation: Likelihood: High. Any participant triggers this with no setup and no role.

Internal preconditions:

1. The market entered `FIXED_TERM` once, which any drawdown above `dustTolerance` does.

External preconditions:

1. The mark ticks down after the release is consumed. One basis point suffices.

Proof of Concept: Save as `test/concrete/FixedTermReStamp.t.sol` and run `forge test --match-path test/concrete/FixedTermReStamp.t.sol -vv`.

```
// SPDX-License-Identifier: UNLICENSED  
pragma solidity ^0.8.28;  
  
import { console2 } from "../../lib/forge-std/src/console2.sol";  
import { IRoycoDayKernel } from "../../src/interfaces/IRoycoDayKernel.sol";  
import { IRoycoVaultTranche } from "../../src/interfaces/IRoycoVaultTranche.sol";  
import { AssetClaims, MarketState } from "../../src/libraries/Types.sol";  
import { toUint256 } from "../../src/libraries/Units.sol";  
import { EntryPointTestBase } from "../utils/EntryPointTestBase.sol";
```



```
import { defaultParams } from "../utils/MarketParams.sol";
import { cellA } from "../utils/TokenConfigs.sol";

/// Proves that a released fixed term is re-stamped at full length by the next
→ ordinary downtick, so total
/// frozen time is unbounded and redemptions stay shut across consecutive terms.
contract FixedTermReStampTest is EntryPointTestBase {
    uint256 internal collateralUnit;
    uint256 internal quoteUnit;
    address internal juniorHolder;
    address internal liquidityProvider;
    address internal caller;

    function setUp() public {
        _deployMarket(cellA(), defaultParams());
        collateralUnit = 10 ** uint256(cell.collateralAsset.decimals);
        quoteUnit = 10 ** uint256(cell.quoteAsset.decimals);
        _seedMarket(100 * collateralUnit, 30 * collateralUnit);
        _seedLPT(20e18, 0, 20 * quoteUnit);
        _deployEntryPoint();
        juniorHolder = _generateEntryPointUser("JUNIOR_HOLDER");
        liquidityProvider = _generateEntryPointUser("LIQUIDITY_PROVIDER");
        caller = makeAddr("UNPRIVILEGED_CALLER");
        vm.deal(caller, 100 ether);
    }

    /// @dev The live timestamp read through a cheatcode, since via_ir may fold a
    → plain block.timestamp
    function _now() internal view returns (uint256) {
        return vm.getBlockTimestamp();
    }

    /// @dev Warps to an absolute timestamp and refreshes both feeds so no staleness
    → gate trips
    function _advanceTo(uint256 _timestamp) internal {
        if (_timestamp > _now()) vm.warp(_timestamp);
        collateralAssetOracle.setUpdatedAt(_now());
        priceFeed.setUpdatedAt(_now());
    }

    /// @dev A sync committed by an address holding no role at all
    function _syncAsCaller() internal {
        vm.prank(caller);
        kernel.syncTrancheAccounting();
    }

    function _setCollateralPrice(uint256 _priceWAD) internal {
        collateralPriceWAD = _priceWAD;
        collateralAssetOracle.setPrice(_priceWAD);
        collateralAssetOracle.setUpdatedAt(_now());
    }

    function _committedState() internal view returns (MarketState) {
        return accountant.getState().lastMarketState;
    }

    /// @dev Walk the mark down by the smallest step that opens a fixed term, so the
    → drawdown stays minimal
    function _openMinimalDrawdown(uint256 _basePriceWAD) internal {
        uint256 stepWAD = 1;
        for (uint256 i; i < 40; ++i) {
            _setCollateralPrice(_basePriceWAD - stepWAD);
            _syncAsCaller();
            if (_committedState() == MarketState.FIXED_TERM) return;
        }
    }
}
```



```
        stepWAD <= 1;
    }
    revert("no dust-sized drawdown opens a fixed term at these marks");
}

/**
 * notice Ordinary traffic consumes the one-shot release, then a 1 bp downtick
 * → twelve seconds later
 * → stamps a FULL fresh term rather than the unserved remainder of the
 * → original one.
 */
function test_ordinaryTrafficConsumesReleaseAndNextDowntickReStampsAFullTerm()
→ public {
    uint256 jtShares = _acquireTrancheShares(juniorHolder,
        → address(juniorTranche), 5 * collateralUnit);

    _openMinimalDrawdown(collateralPriceWAD);
    uint32 termEnd = accountant.getState().fixedTermEndTimestamp;
    uint256 termDuration = accountant.getState().fixedTermDurationSeconds;

    _advanceTo(uint256(termEnd) + 1);

    // Ordinary traffic, not an attack: liquidity provider deposits are the one
    → operation allowed in
    // FIXED_TERM, so a provider with no knowledge of the term burns the release
    → for everyone else.
    _acquireTrancheShares(liquidityProvider, address(liquidityProviderTranche),
        → 1e18);
    assertEq(uint8(_committedState()), uint8(MarketState.PERPETUAL), "the
        → release was consumed");

    // One block later the mark ticks down 1 bp. Nothing was manipulated.
    _advanceTo(_now() + 12);
    _setCollateralPrice(collateralPriceWAD * 9999 / 10_000);

    vm.prank(juniorHolder);
    vm.expectRevert(IRoycoDayKernel.DISABLED_IN_FIXED_TERM_STATE.selector);
    IRoycoVaultTranche(address(juniorTranche)).redeem(jtShares, juniorHolder,
        → juniorHolder);

    _syncAsCaller();
    assertEq(uint8(_committedState()), uint8(MarketState.FIXED_TERM), "re-locked
        → with no attacker involved");
    assertEq(uint256(accountant.getState().fixedTermEndTimestamp), _now() +
        → termDuration, "a FULL fresh term");
    console2.log("original term end", uint256(termEnd));
    console2.log("fresh term end ",
        → uint256(accountant.getState().fixedTermEndTimestamp));
}

/**
 * notice CONTROL. Holds the price path identical and varies ONLY whether the
 * → release is consumed.
 * → Uncontested the holder exits; with the release consumed the same
 * → redemption reverts.
 */
function test_control_consumingTheReleaseFlipsAWorkingRedeemIntoARevert()
→ public {
    uint256 jtShares = _acquireTrancheShares(juniorHolder,
        → address(juniorTranche), 5 * collateralUnit);

    uint256 basePriceWAD = collateralPriceWAD;
    _openMinimalDrawdown(basePriceWAD);
    uint32 termEnd = accountant.getState().fixedTermEndTimestamp;
```



```
_advanceTo(uint256(termEnd) + 1);

// BRANCH A: nobody consumes the release. A 3% drawdown lands, then the
↳ holder redeems. Its own pre-op
// sync takes the release and the exit succeeds despite the drawdown.
uint256 snapA = vm.snapshotState();
_advanceTo(_now() + 30 minutes);
_setCollateralPrice(basePriceWAD * 97 / 100);
vm.prank(juniorHolder);
AssetClaims memory ok =
↳ IRoycoVaultTranche(address(juniorTranche)).redeem(jtShares,
↳ juniorHolder, juniorHolder);
assertGt(toUint256(ok.collateralAssets), 0, "uncontested, the holder exits
↳ through its own release sync");
require(vm.revertToState(snapA), "unwind");

// BRANCH B: one bare unprivileged sync consumes the release. The SAME
↳ drawdown at the SAME instant
// now stamps a fresh term and the SAME redemption reverts.
_syncAsCaller();
_advanceTo(_now() + 30 minutes);
_setCollateralPrice(basePriceWAD * 97 / 100);
vm.prank(juniorHolder);
vm.expectRevert(IRoycoDayKernel.DISABLED_IN_FIXED_TERM_STATE.selector);
IRoycoVaultTranche(address(juniorTranche)).redeem(jtShares, juniorHolder,
↳ juniorHolder);
assertEq(juniorTranche.maxRedeem(juniorHolder), 0, "the exit is shut for the
↳ holder that would have escaped");
}

/// @notice Ten consecutive terms on an ordinary decline. Each new low re-stamps
↳ a full term.
function test_tenConsecutiveTermsHoldTheMarketShut() public {
    _acquireTrancheShares(juniorHolder, address(juniorTranche), 5 *
↳ collateralUnit);

    uint256 basePriceWAD = collateralPriceWAD;
    _openMinimalDrawdown(basePriceWAD);

    uint256 termDuration = accountant.getState().fixedTermDurationSeconds;
    uint32 originalEnd = accountant.getState().fixedTermEndTimestamp;
    uint32 termEnd = originalEnd;
    uint256 mark = collateralPriceWAD;

    for (uint256 i; i < 10; ++i) {
        _advanceTo(uint256(termEnd) + 1);
        _syncAsCaller();
        // One block later the mark makes a new low: 5 bps down, a completely
        ↳ ordinary move.
        _advanceTo(_now() + 12);
        mark = mark * 9995 / 10_000;
        _setCollateralPrice(mark);
        _syncAsCaller();
        termEnd = accountant.getState().fixedTermEndTimestamp;
        assertEq(uint8(_committedState()), uint8(MarketState.FIXED_TERM), "each
        ↳ new low re-locks");
        assertEq(uint256(termEnd), _now() + termDuration, "each re-lock is a
        ↳ FULL fresh term");
    }

    assertEq(juniorTranche.maxRedeem(juniorHolder), 0, "the junior exit never
    ↳ reopened");
    console2.log("seconds of freeze added", uint256(termEnd) -
    ↳ uint256(originalEnd));
}
```



```

        console2.log("total drawdown (bps)    ", 10_000 - (mark * 10_000 /
        ↪ basePriceWAD));
    }
}

```

```

[PASS] test_control_consumingTheReleaseFlipsAWorkingRedeemIntoARevert() (gas:
↪ 1205964)
[PASS] test_ordinaryTrafficConsumesReleaseAndNextDowntickReStampsAFullTerm() (gas:
↪ 1137062)
    original term end 1209601
    fresh term end    2419214
[PASS] test_tenConsecutiveTermsHoldTheMarketShut() (gas: 2240077)
    seconds of freeze added 12096130
    total drawdown (bps)    50
Suite result: ok. 3 passed; 0 failed; 0 skipped

```

The first test asserts the re-stamped end equals `now + termDuration`, a full term not the remainder. The control holds the price path identical, varying only whether the release is consumed, flipping a working redeem into a revert. The ten-term test adds 12,096,130 seconds of freeze on a 50 bp decline.

Recommendation: Carry the unserved remainder instead of starting a new term. Record the episode's end where condition 5 clears `fixedTermEndTimestamp` and resume it at `RoycoDayAccountant.sol:561` while still future:

```

if (initialMarketState == MarketState.PERPETUAL) {
    uint32 carried = $.fixedTermEpisodeEndTimestamp;
    fixedTermEndTimestamp = carried > block.timestamp ? carried :
    ↪ uint32(block.timestamp + fixedTermDurationSeconds);
}

```

Clear the field once the junior impermanent loss is repaid within `dustTolerance`, so a new drawdown earns a full period. Persist it in the committing caller, since `_previewSyncTrancheAccounting` is `view`.

Royco: Acknowledged. A full fresh term per episode is the intended semantics: each drawdown episode receives its own full observation window, as the fixed term is the recovery period for that episode's impermanent loss claim.

Cantina Managed: Acknowledged.